

MACHINE LEARNING IN FINANCE

MODULE 3 | LESSON 2

BAYESIAN STATISTICS IN PRACTICE

Reading Time	15 minutes
Prior Knowledge	Naïve Bayes classifier methodology, likelihood, prior.
Keywords	Gaussian Distributions, Model fitting, Likelihood, Prior distribution

In the previous lesson, we looked at how Bayesian statistics can be used to assign the probability of an outcome occurring. We looked at categorical features and will now look at continuous features to predict the future one-month return of a group of stocks exceeding the median.

1. Naïve Bayes in Practice

Recall from Module 3, Lesson 1, that posterior distribution can be expressed as a product of the likelihoods of the predictors given the class and the prior distributions of the class, as in (1.4). This is based on the assumption of conditional independence. This is true for predictors that are numeric and continuous as well since these predictors also have likelihood distributions given a specific target class.

$$\begin{aligned} P(Y=k \mid X_{\{1\}}, X_{\{2\}}, \cdots, X_{\{n\}}) &= \frac{P(X_{\{1\}}|Y=k)P(X_{\{2\}}|Y=k)\cdots P(X_{\{n\}}|Y=k)P(Y=k)}{\sum_j P(Y=j)\prod_{i=1}^n P(X_{\{i\}}|Y=j)} \tag{2.1} \end{aligned}$$

Let's consider a case where the likelihood distribution of the predictor follows a Gaussian distribution for each class. The predictor would be a strong explanatory variable, should the distributions differ. A strong predictor, which is often what we aim for, has very little overlap between the likelihoods for each class, thus differentiating between classes. This is shown in figure 1 for both a strong and weak predictor. The top plot shows the likelihood for a strong predictor given the target class as there is little overlap between the distributions (the two curves). The bottom plot shows a weaker predictor with significantly more overlap.

Fig 1: Strong vs. Weak Predictor Likelihoods

To put these ideas into perspective, let us look at an example where we apply the Naïve Bayes classifier to a stock returns problem.

In this section, we use data from Coqueret and Guida (chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data. This data can be accessed via <https://github.com/shokru/mlfactor.github.io/tree/master/material>. For illustrative purposes, we take a subset of stocks and features to predict the future one-month price being above the median, thus a classification problem. The features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*
- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We perform the analysis using the Python statistical software. Let us load the necessary libraries and import the data. We'll work with data from the year 2000 to 2018 and only 50 stock IDs.

```
In [ ]: import numpy as np
import pandas as pd

df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]
```

The objective is to predict whether the future one-month returns are above the median. Hence, we can simply flag instances when it is above the median as 1 and 0 otherwise.

```
In [ ]: # Created a class variable for 1-Month returns if > median

df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"] > df_50["R1M_Usd"].median(), 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)
```

Specify the predictors or features used in the classifier. These features have been transformed into variables that follow a uniform distribution. In practice, we can hope for Gaussian-distributed variables. Therefore, we will transform each feature using the Box-Muller transformation (Box and Muller 610).

```

In [ ]: # create a copy of data to store Gaussian converted data

df50_norm = df_50.copy()

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe"]

# Loop through Box-Muller for each data point for each predictor
for col in feature_cols:
    for i in np.arange(len(df_50)):
        # 1. Use U1 as current data value. Generate U2, which is Unif(0, 1)

        u1s, u2s = (
            df_50.loc[i, col],
            np.random.uniform(low=0.0, high=1.0, size=1)[0],
        ) # X.loc[i, col]

        # 2. Transform U1 to s

        ss = -np.log(u1s)

        # 3. Transform U2 to theta

        thetas = 2 * np.pi * u2s

        # 4. Convert s to r

        rs = np.sqrt(2 * ss)

        # 5. Calculate x and y from r and theta

        xs, ys = rs * np.cos(thetas), rs * np.sin(thetas)

        # 6. Store only one of the Gaussian derived values

        df50_norm.loc[i, col] = ys

```

It is worth looking at the likelihoods given the target class to gauge how strong or weak we can expect the predictors to be. We do this using the code below:

```

In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(2, 3, figsize=(18, 10), sharey=True)

i = 0

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 1

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 2

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 0

```

```

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C
i = 1

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C
i = 2

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C

```

Fig. 2: Likelihood for All 6 Features Given the Respective Target Class

Looking at the output in figure 2 and comparing it to figure 1, we can see the predictive power of the features are quite weak considering the significant overlap in the likelihoods. We can therefore expect poor model performance of the classifier.

We then load the library needed to split the dataset into a training and testing dataset to train and evaluate the classifier, respectively. An 80/20 train/test split is used. We consider only the features mentioned above.

```

In [ ]: # split dataset in features and target variable

X = df50_norm[feature_cols] # Transformed Features

y = df50_norm["R1M_Usd_C"] # Target variable

# Split dataset into training set and test set 80/20

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Let us fit the Naïve Bayes classifier on the training set and obtain predictions on the unseen test set.

```

In [ ]: # initialize and fit classifier to training set
from sklearn.naive_bayes import GaussianNB

classifier = GaussianNB()

classifier.fit(X_train, y_train)

# get the predictions

y_pred = classifier.predict(X_test)

# Extract the probabilities of Target Class = 1.

y_predprobs = classifier.predict_proba(X_test)

probs = y_predprobs[:, 1]

# print the accuracy

```

```

from sklearn import metrics

# Model Accuracy, how often is the classifier correct?

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

```

The ROC curve and AUC can be obtained using the code below. It is useful to also show a no-skill or random guess classifier in the plot to compare to the Naïve Bayes classifier.

```

In [ ]: # plot the roc curve for the model
from sklearn.metrics import roc_auc_score, roc_curve

# generate a no skill or random guess prediction

ns_probs = [0 for _ in range(len(y_test))]

# calculate roc curves

ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

# calculate scores

ns_auc = roc_auc_score(y_test, ns_probs) # random guess

tree_auc = roc_auc_score(y_test, probs) # tree classifier

# summarize scores

print("No Skill: ROC AUC=%.3f" % (ns_auc))

print("Classifier: ROC AUC=%.3f" % (tree_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", linewidth=4, label="No Skill")

plt.plot(
    ns_fpr,
    ns_tpr,
    linewidth=2,
    label="NB Classifier",
)

# axis labels

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

# show the legend

plt.legend()

# show the plot

plt.show()

```

Fig. 3: ROC Curve of Naïve Bayes Classifier Performance along with a Random Guess or No-Skill Model

The corresponding ROC curve is shown in figure 3. The AUC for this model is ~50%, thus confirming our suspicion of its weak predictive power.

2. Conclusion

The example covered in this lesson helps us understand the Naïve Bayes classifier by seeing it in practice. The model performance, however, is not as good as we would hope. This lesson focused on showing how to apply Naïve Bayes to a real-world classification problem using continuous predictors and how Naïve Bayes can easily be applied to categorical variables. To obtain a better model, stronger predictors are needed, i.e., explanatory variables that have different likelihood distributions for the target classes. Explanatory data analysis, feature selection, and engineering cannot be understated when dealing with modeling.

In the next section, we will look at tree-based algorithms, another modeling technique. The Naïve Bayes model assumes the features are conditionally independent of the target variable; however, tree-based algorithms do not require any assumptions about the underlying distribution. Both Naïve Bayes and trees do not require feature scaling, although tree-based algorithms are not sensitive to outliers. The prediction approach of trees are to place decision boundaries in the data whereas Naïve Bayes aims to model how the data were generated. These are known as discriminative and generative models, respectively. We will look at trees in depth in the next lesson.

References

- Box, G. E. P., and Mervin E. Muller. "A Note on the Generation of Random Normal Deviates." *The Annals of Mathematical Statistics*, vol. 29, no. 2, 1958, pp. 610–11, <https://doi.org/10.1214/aoms/1177706645>.
- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. Financial Mathematics Series. Chapman and Hall/CRC, 2020.

MACHINE LEARNING IN FINANCE

MODULE 3 | LESSON 4

TREES IN PRACTICE

Reading Time 12 minutes

Prior Knowledge Tree-based machine learning methodology

Keywords Model fitting, Training models, Testing models, Model performance, ROC, Area Under the Curve

In the previous lesson, we looked at how tree-based algorithms can be used to predict class outcomes. We now apply this theory to predict one-month returns of a group of stocks exceeding the median or not.

1. Trees in Practice

Recall from Module 3, Lesson 3 that given a set of data with predictors and a target variable with class outcomes, we can fit a decision tree classifier that can intuitively show the rules or criteria that makes a target class highly likely. This non-black box characteristic is a benefit of decision trees as we will show in this lesson by fitting a tree to stock data in order to predict whether a future one-month return will change by more than a threshold or not, i.e., we consider the same binary class problem in lesson 2 of this module.

We therefore use the same data (from Coqueret and Guida chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data.

2. The Data

The data can be accessed via

<https://github.com/shokru/mlfactor.github.io/tree/master/material>. Recall that the features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*

- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We use Python to perform the analysis and take the same data from the year 2000 to 2018 with only 50 stock IDs. Let us read in the data as in Lesson 2 and take a preview.

```
In [1]: import numpy as np
import pandas as pd

# Location of the data

# ENTER LOCATION OF DATA FILE BELOW
df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]

# Created a class variable for 1-Month returns if
# changes by more than a threshold = thresh

thresh = 0.04
df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"].abs() > thresh, 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe"

df_50 = df_50[feature_cols + ["R1M_Usd_C"]]

df_50.head()
```

```
Out[1]:
```

	Mkt_Cap_12M_Usd	Pb	Vol1Y_Usd	Return_On_Capital	Roe	Pe	R1M_Usd_C
0	0.52	0.21	0.77	0.08	0.02	0.5	0
1	0.52	0.21	0.73	0.08	0.02	0.5	1
2	0.51	0.15	0.67	0.10	0.06	0.5	0
3	0.51	0.15	0.63	0.10	0.06	0.5	1
4	0.52	0.15	0.54	0.10	0.06	0.5	1

We see 6 predictors with the target variable being the one-month future returns (R1M_Used_C), that is a binary variable taking on 1 if the return is more than 4% in either a long or short direction and 0 otherwise. Keep in mind that features have been pre-selected in this example. In practice, however, feature selection generally takes a significant time to perform and often adds much value in improving performance.

3. Train the Tree

Now that we have a view of the data, we will proceed with splitting the dataset into a train/test 80/20 split.

```
In [2]: from sklearn.model_selection import train_test_split

# split dataset in features and target variable
X = df_50[feature_cols] # Transformed Features

y = df_50["R1M_Used_C"] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

We now declare our decision tree classifier using the `sklearn` package.

```
In [3]: # Import Decision Tree Classifier

from sklearn.tree import DecisionTreeClassifier

# Create Decision Tree classifier object

clf = DecisionTreeClassifier(criterion="gini", max_depth=3, min_samples_leaf=25)

# Train Decision Tree classifier

clf = clf.fit(X_train, y_train)
```

The decision tree produces a set of rules that are easily interpretable in our case since we have used only a few features for illustrative purposes. A flow diagram of the decision tree can be shown using the code below, which will produce a flow diagram as in Figure 1. This shows that companies with turnover above 0.905 are 78% likely to have returns that change by more than 4%. This is a strong differentiating node.

```
In [4]: # visualize the tree

import matplotlib.pyplot as plt
from sklearn import tree

# features we used

fn = feature_cols

# labels of the target class

cn = ["0", "1"]
```

```
fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(clf, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagename.png')
fig.savefig("decistion_tree.png")
```

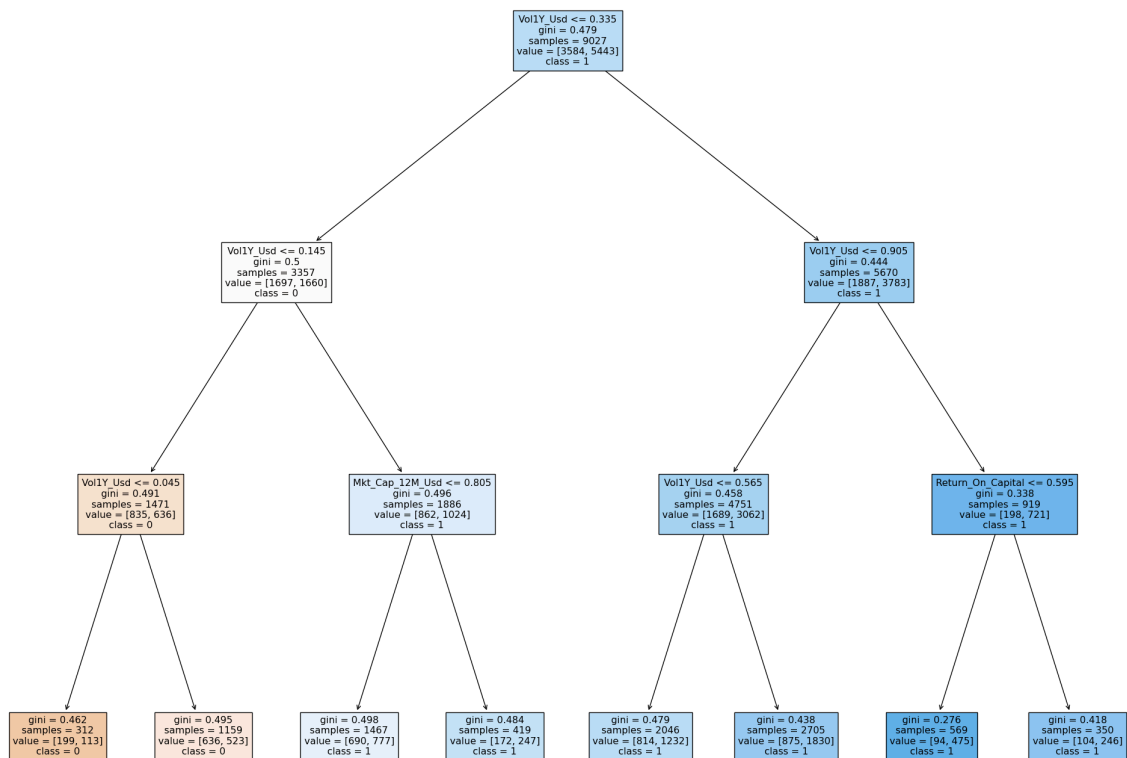


Fig. 1: Flow Diagram of Decision Tree for Predicting Price Change of More than 4% in Any Direction

Figure 1 shows an absolute change exceeding 4% as class 1 and class 0 otherwise. The blue nodes represent cases that majority have class 1, i.e., the darker the shade of blue, the purer the node. Similarly, for Class 0, the more orange the node, the higher proportion of Class 0.

It is worth pointing out that the cuts in Trees are parallel to existing axis whereas in SVM the cuts can be, but not restricted to, a linear combination of features. The sequential nature of the splitting shown in Fig 1 shows that only one feature can be used at a time. SVM on the other hand is not restricted by this.

4. Tree Performance

We've trained our decision tree classifier and we are now ready to test the performance. To do this, we need to run the unseen test dataset through the classifier to determine the

class predictions for the target variable. It would be insightful to compare this to a random guess classifier, i.e., a model that requires no skill.

In [5]: *# Predict the response for test dataset*

```
y_pred = clf.predict(X_test)

# Obtain the probabilities
probs_tmp = clf.predict_proba(X_test)

probs = probs_tmp[:, 1]
```

Fortunately, `Sklearn` has tools to evaluate performance metrics for most machine learning algorithms. We look at the ROC curve and AUC as in Lesson 2.

In [6]: **import** matplotlib.pyplot **as** plt
from sklearn.metrics **import** roc_auc_score, roc_curve

```
# plot the roc curve for the model

# generate a no skill or random guess prediction

ns_probs = [0 for _ in range(len(y_test))]

# calculate roc curves for random guess

ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

# calculate roc curves for the decision tree

fpr, tpr, _ = roc_curve(y_test, probs)

# calculate scores

ns_auc = roc_auc_score(y_test, ns_probs) # random guess

tree_auc = roc_auc_score(y_test, probs) # tree classifier

# summarize scores

print("No Skill: ROC AUC=%.3f" % (ns_auc))

print("Classifier: ROC AUC=%.3f" % (tree_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")

plt.plot(fpr, tpr, marker=".", label="Tree Classifier")

# axis labels

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

# show the legend

plt.legend()
```

```
# show the plot
```

```
plt.show()
```

No Skill: ROC AUC=0.500

Classifier: ROC AUC=0.606

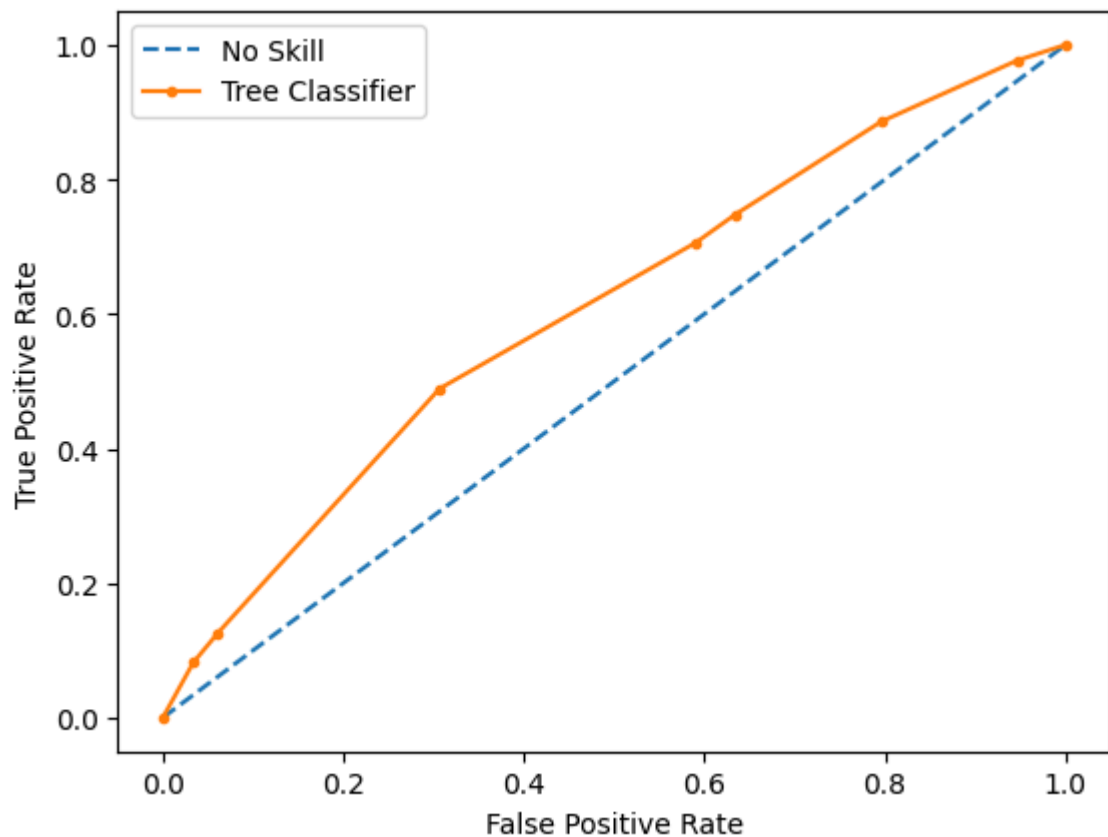


Fig. 2: ROC Curve of the Decision Tree Classifier vs. a Random Guess or No Skill Predictor

The AUC for the classifier is 60.6%, which is better than a random guess. It is worth noting that a subset of predictors was chosen to illustrate the decision tree flow diagram in Figure 1. Feature selection plays an important part in model performance and should not be overlooked.

Lastly, it is worth looking at the implementation of a tree-based algorithm that uses boosting. The algorithm we consider is popular in the Kaggle community and can be easily implemented in Python using the `xgboost` package. The hyperparameters have been specified, but not optimized, instead of the default settings. For further reading on these parameters, please refer to ("[XGBoost Parameters](#) — [Xgboost 1.6.1 Documentation](#)")

```
In [7]: import xgboost as xgb

# Init classifier
xgb_cl = xgb.XGBClassifier(
    max_depth=2,
    eta=0.1, # Learning rate
    objective="binary:logistic",
    eval_metric="auc",
)
```

```

# Fit
xgb_cl.fit(X_train, y_train)

# Predict
preds_xgb = xgb_cl.predict(X_test)

# Obtain the probabilities
preds_prob_xgb_tmp = xgb_cl.predict_proba(X_test)

preds_prob_xgb = preds_prob_xgb_tmp[:, 1]

```

The results of the `xgboost` model along with the decision tree and random guess classifier is shown in figure 3 below. The `XGBoost` ties with the decision tree at 60.6%. The reader, as an exercise, can decrease the max depth of the tree to determine the change in performance. This will show the sensitivity to the hyperparameters.

```

In [8]: # generate a no skill or random guess prediction
ns_probs = [0 for _ in range(len(y_test))]
# calculate roc curves
# random
ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)
# tree
fpr, tpr, _ = roc_curve(y_test, probs)
# xgb
xgb_fpr, xgb_tpr, _ = roc_curve(y_test, preds_prob_xgb)

# calculate scores
ns_auc = roc_auc_score(y_test, ns_probs) # random guess
tree_auc = roc_auc_score(y_test, probs) # tree classifier
xgb_auc = roc_auc_score(y_test, preds_prob_xgb) # tree classifier
# summarize scores
print("No Skill: ROC AUC=%.3f" % (ns_auc))
print("Tree Clasifier: ROC AUC=%.3f" % (tree_auc))
print("XGB Clasifier: ROC AUC=%.3f" % (xgb_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")
plt.plot(fpr, tpr, marker=".", label="Tree Classifier")
plt.plot(xgb_fpr, xgb_tpr, marker=".", label="XGB Classifier")
# axis labels
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# show the legend
plt.legend()
# show the plot
plt.show()

```

```

No Skill: ROC AUC=0.500
Tree Clasifier: ROC AUC=0.606
XGB Clasifier: ROC AUC=0.606

```

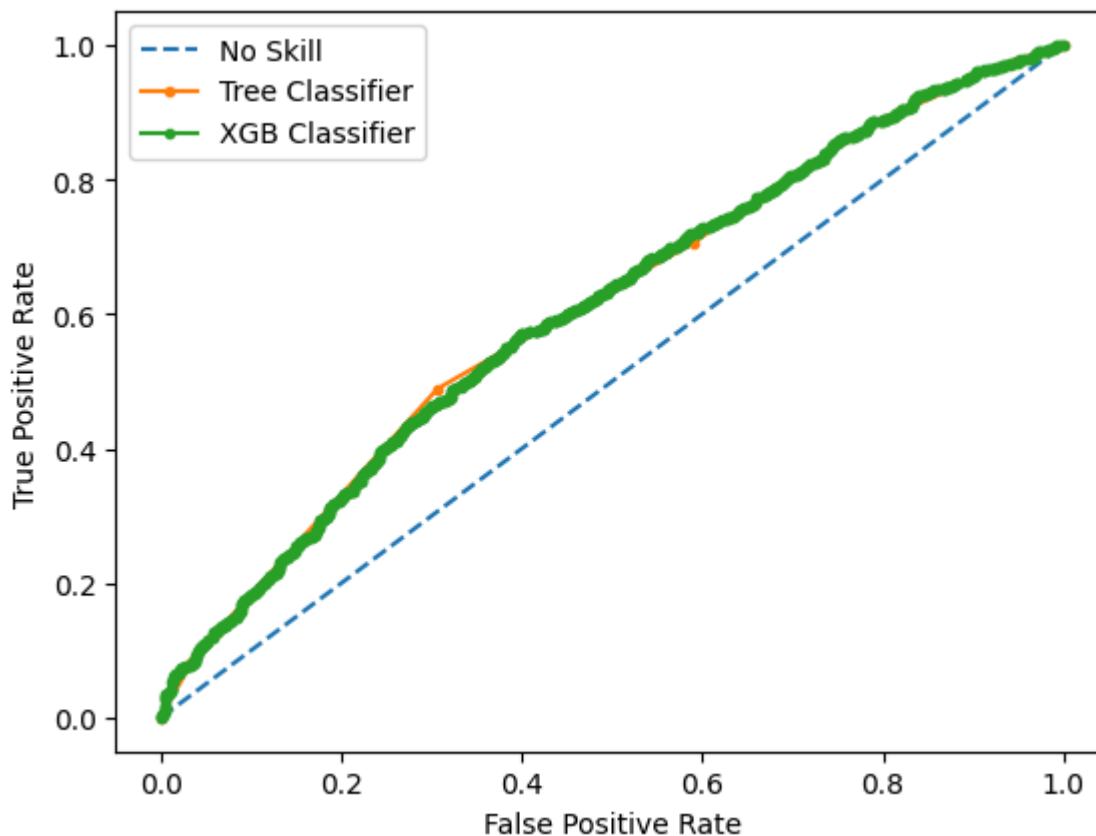


Fig. 3: ROC Curves for the Tree, XGBoost, and No-Skill Model

5. Conclusion

The examples illustrated in this lesson help us understand the tree-based algorithms from the simple decision tree to the `xgboost` model. The model performance is better than a random guess thus showing added benefit. This lesson focused on showing how to apply tree-based algorithms to a real-world classification problem. A model that does better than a random guess can be easily motivated; however, keep in mind the goal of the specific classification problem, as it may require a minimum AUC to be achieved. The next module will explore support vector machines and neural networks, another class of predictive models.

References

- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. CRC Financial Mathematics Series, Chapman and Hall/CRC, 2022.
- "XGBoost Parameters — Xgboost 1.6.1 Documentation." XGBoost Parameters, <https://xgboost.readthedocs.io/en/stable/parameter.html>. Accessed 30 June 2022.

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and Amazon.

✓ Classification

```
# install empiricaldist if necessary

try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py

from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)

download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.p
```

```
from utils import set_pyplot_params

set_pyplot_params()
```

```
from utils import Or70, Pu50, Gr30

color_list3 = [Or70, Pu50, Gr30]
```

```
import matplotlib.pyplot as plt
from cycler import cycler

marker_cycle = cycler(marker=['s', 'o', '^'])
color_cycle = cycler(color=color_list3)
line_cycle = cycler(linestyle=['-', '--', ':'])

plt.rcParams['axes.prop_cycle'] = (color_cycle +
                                   marker_cycle +
                                   line_cycle)
```

Classification might be the most well-known application of Bayesian methods, made famous in the 1990s as the basis of the first generation of [spam filters](#).

In this chapter, I'll demonstrate Bayesian classification using data collected and made available by Dr. Kristen Gorman at the Palmer Long-Term Ecological Research Station in Antarctica (see Gorman, Williams, and Fraser, "[Ecological Sexual Dimorphism and Environmental Variability within a Community of Antarctic Penguins \(Genus *Pygoscelis*\)](#)", March 2014). We'll use this data to classify penguins by species.

The following cell downloads the raw data.

```
# Load the data files from
# https://github.com/allisonhorst/palmerpenguins
# With gratitude to Allison Horst (@allison_horst)

download('https://github.com/allisonhorst/palmerpenguins/raw/main/inst/extda
```

✓ Penguin Data

I'll use Pandas to load the data into a `DataFrame`.

```
import pandas as pd

df = pd.read_csv('penguins_raw.csv').dropna(subset=['Body Mass (g)'])
df.shape
```

The dataset contains one row for each penguin and one column for each variable.

```
df.head()
```

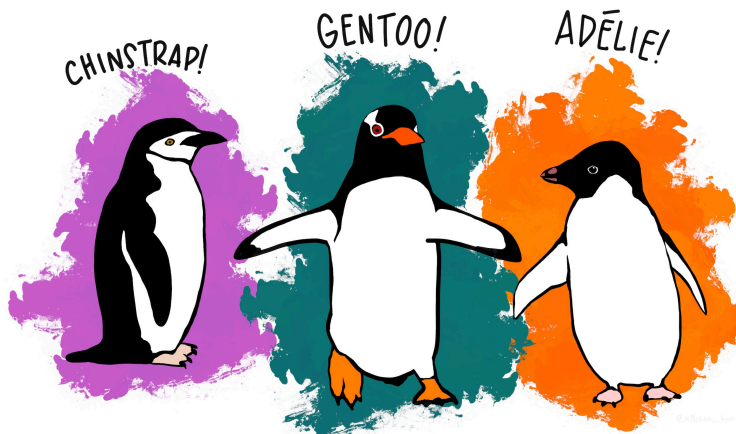
For convenience, I'll create a new column called `Species2` that contains a shorter version of the species names.

```
def shorten(species):
    return species.split()[0]

df['Species2'] = df['Species'].apply(shorten)
```

Three species of penguins are represented in the dataset: Adélie, Chinstrap and Gentoo.

These species are shown in this illustration (by Allison Horst, available under the [CC-BY](#) license):

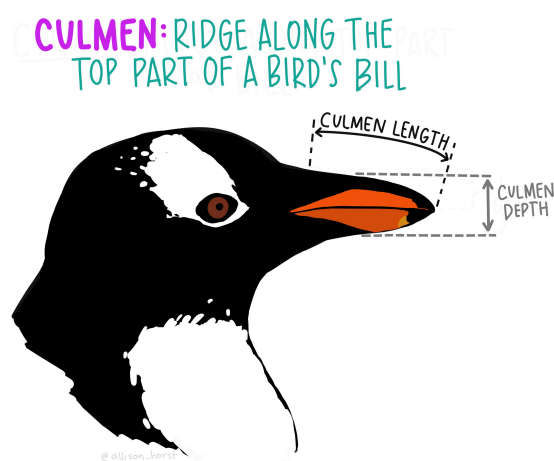


The measurements we'll use are:

- Body Mass in grams (g).
- Flipper Length in millimeters (mm).
- Culmen Length in millimeters.
- Culmen Depth in millimeters.

If you are not familiar with the word "culmen", it refers to the [top margin of the beak](#).

The culmen is shown in the following illustration (also by Allison Horst):



These measurements will be most useful for classification if there are substantial differences between species and small variation within species. To see whether that is true, and to what degree, I'll plot cumulative distribution functions (CDFs) of each measurement for each species.

The following function takes the `DataFrame` and a column name. It returns a dictionary that maps from each species name to a `Cdf` of the values in the column

named `colname`.

```
def make_cdf_map(df, colname, by='Species2'):
    """Make a CDF for each species."""
    cdf_map = {}
    grouped = df.groupby(by)[colname]
    for species, group in grouped:
        cdf_map[species] = Cdf.from_seq(group, name=species)
    return cdf_map
```

The following function plots a `Cdf` of the values in the given column for each species:

```
from empiricaldist import Cdf
from utils import decorate

def plot_cdfs(df, colname, by='Species2'):
    """Make a CDF for each species.

    df: DataFrame
    colname: string column name
    by: string column name

    returns: dictionary from species name to Cdf
    """
    cdf_map = make_cdf_map(df, colname, by)

    for species, cdf in cdf_map.items():
        cdf.plot(label=species, marker='')

    decorate(xlabel=colname,
            ylabel='CDF')
```

Here's what the distributions look like for culmen length.

```
colname = 'Culmen Length (mm)'
plot_cdfs(df, colname)
```

It looks like we can use culmen length to identify Adélie penguins, but the distributions for the other two species almost entirely overlap.

Here are the distributions for flipper length.

```
colname = 'Flipper Length (mm)'
plot_cdfs(df, colname)
```

Using flipper length, we can distinguish Gentoo penguins from the other two species. So with just these two features, it seems like we should be able to classify penguins with some accuracy.

All of these CDFs show the sigmoid shape characteristic of the normal distribution; I will take advantage of that observation in the next section.

Here are the distributions for culmen depth.

```
colname = 'Culmen Depth (mm)'\nplot_cdfs(df, colname)
```

And here are the distributions of body mass.

```
colname = 'Body Mass (g)'\nplot_cdfs(df, colname)
```

Culmen depth and body mass distinguish Gentoo penguins from the other two species, but these features might not add a lot of additional information, beyond what we get from flipper length and culmen length.

✓ Normal Models

Let's use these features to classify penguins. We'll proceed in the usual Bayesian way:

1. Define a prior distribution with the three possible species and a prior probability for each,
2. Compute the likelihood of the data for each hypothetical species, and then
3. Compute the posterior probability of each hypothesis.

To compute the likelihood of the data under each hypothesis, I'll use the data to estimate the parameters of a normal distribution for each species.

The following function takes a `DataFrame` and a column name; it returns a dictionary that maps from each species name to a `norm` object.

`norm` is defined in SciPy; it represents a normal distribution with a given mean and standard deviation.

```
from scipy.stats import norm\n\ndef make_norm_map(df, colname, by='Species2'):\n    """Make a map from species to norm object."""\n    norm_map = {}
```

```
grouped = df.groupby(by)[colname]
for species, group in grouped:
    mean = group.mean()
    std = group.std()
    norm_map[species] = norm(mean, std)
return norm_map
```

For example, here's the dictionary of `norm` objects for flipper length:

```
flipper_map = make_norm_map(df, 'Flipper Length (mm)')
flipper_map.keys()
```

Now suppose we measure a penguin and find that its flipper is 193 cm. What is the probability of that measurement under each hypothesis?

The `norm` object provides `pdf`, which computes the probability density function (PDF) of the normal distribution. We can use it to compute the likelihood of the observed data in a given distribution.

```
data = 193
flipper_map['Adelie'].pdf(data)
```

The result is a probability density, so we can't interpret it as a probability. But it is proportional to the likelihood of the data, so we can use it to update the prior.

Here's how we compute the likelihood of the data in each distribution.

```
hypos = flipper_map.keys()
likelihood = [flipper_map[hypo].pdf(data) for hypo in hypos]
likelihood
```

Now we're ready to do the update.

✓ The Update

As usual I'll use a `Pmf` to represent the prior distribution. For simplicity, let's assume that the three species are equally likely.

```
from empiricaldist import Pmf

prior = Pmf(1/3, hypos)
prior
```

Now we can do the update in the usual way.

```
posterior = prior * likelihood
posterior.normalize()
posterior
```

A penguin with a 193 mm flipper is unlikely to be a Gentoo, but might be either an Adélie or Chinstrap (assuming that the three species were equally likely before the measurement).

The following function encapsulates the steps we just ran. It takes a `Pmf` representing the prior distribution, the observed data, and a map from each hypothesis to the distribution of the feature.

```
def update_penguin(prior, data, norm_map):
    """Update hypothetical species."""
    hypos = prior.qs
    likelihood = [norm_map[hypo].pdf(data) for hypo in hypos]
    posterior = prior * likelihood
    posterior.normalize()
    return posterior
```

The return value is the posterior distribution.

Here's the previous example again, using `update_penguin`:

```
posterior1 = update_penguin(prior, 193, flipper_map)
posterior1
```

As we saw in the CDFs, flipper length does not distinguish strongly between Adélie and Chinstrap penguins.

But culmen length *can* make this distinction, so let's use it to do a second round of classification. First we estimate distributions of culmen length for each species like this:

```
culmen_map = make_norm_map(df, 'Culmen Length (mm)')
```

Now suppose we see a penguin with culmen length 48 mm. We can use this data to update the prior.

```
posterior2 = update_penguin(prior, 48, culmen_map)
posterior2
```

A penguin with culmen length 48 mm is about equally likely to be a Chinstrap or Gentoo.

Using one feature at a time, we can often rule out one species or another, but we generally can't identify species with confidence. We can do better using multiple features.

✓ Naive Bayesian Classification

To make it easier to do multiple updates, I'll use the following function, which takes a prior `Pmf`, a sequence of measurements and a corresponding sequence of dictionaries containing estimated distributions.

```
def update_naive(prior, data_seq, norm_maps):
    """Naive Bayesian classifier

    prior: Pmf
    data_seq: sequence of measurements
    norm_maps: sequence of maps from species to distribution

    returns: Pmf representing the posterior distribution
    """
    posterior = prior.copy()
    for data, norm_map in zip(data_seq, norm_maps):
        posterior = update_penguin(posterior, data, norm_map)
    return posterior
```

It performs a series of updates, using one variable at a time, and returns the posterior `Pmf`.

To test it, I'll use the same features we looked at in the previous section: culmen length and flipper length.

```
colnames = ['Flipper Length (mm)', 'Culmen Length (mm)']
norm_maps = [flipper_map, culmen_map]
```

Now suppose we find a penguin with flipper length 193 mm and culmen length 48. Here's the update:

```
data_seq = 193, 48
posterior = update_naive(prior, data_seq, norm_maps)
posterior
```

It is almost certain to be a Chinstrap.

```
posterior.max_prob()
```

We can loop through the dataset and classify each penguin with these two features.

```
import numpy as np

df['Classification'] = "None"

for i, row in df.iterrows():
    data_seq = row[colnames]
    posterior = update_naive(prior, data_seq, norm_maps)
    df.loc[i, 'Classification'] = posterior.max_prob()
```

This loop adds a column called `Classification` to the `DataFrame`; it contains the species with the maximum posterior probability for each penguin.

So let's see how many we got right.

```
len(df)
```

```
valid = df['Classification'].notna()
valid.sum()
```

```
same = df['Species2'] == df['Classification']
same.sum()
```

There are 344 penguins in the dataset, but two of them are missing measurements, so we have 342 valid cases. Of those, 324 are classified correctly, which is almost 95%.

```
same.sum() / valid.sum()
```

The following function encapsulates these steps.

```
def accuracy(df):
    """Compute the accuracy of classification."""
    valid = df['Classification'].notna()
    same = df['Species2'] == df['Classification']
    return same.sum() / valid.sum()
```

The classifier we used in this section is called "naive" because it ignores correlations between the features. To see why that matters, I'll make a less naive classifier: one that takes into account the joint distribution of the features.

✓ Joint Distributions

I'll start by making a scatter plot of the data.

```
import matplotlib.pyplot as plt

def scatterplot(df, var1, var2):
    """Make a scatter plot."""
    grouped = df.groupby('Species2')
    for species, group in grouped:
        plt.plot(group[var1], group[var2],
                 label=species, ls="None", alpha=0.3)

    decorate(xlabel=var1, ylabel=var2)
```

Here's a scatter plot of culmen length and flipper length for the three species.

```
var1 = 'Flipper Length (mm)'
var2 = 'Culmen Length (mm)'
scatterplot(df, var1, var2)
```

Within each species, the joint distribution of these measurements forms an oval shape, at least roughly. The orientation of the ovals is along a diagonal, which indicates that there is a correlation between culmen length and flipper length.

If we ignore these correlations, we are assuming that the features are independent. To see what that looks like, I'll make a joint distribution for each species assuming independence.

The following function makes a discrete **Pmf** that approximates a normal distribution.

```
def make_pmf_norm(dist, sigmas=3, n=101):
    """Make a Pmf approximation to a normal distribution."""
    mean, std = dist.mean(), dist.std()
    low = mean - sigmas * std
    high = mean + sigmas * std
    qs = np.linspace(low, high, n)
    ps = dist.pdf(qs)
    pmf = Pmf(ps, qs)
    pmf.normalize()
    return pmf
```

We can use it, along with **make_joint**, to make a joint distribution of culmen length and flipper length for each species.

```
from utils import make_joint

joint_map = {}
for species in hypos:
    pmf1 = make_pmf_norm(flipper_map[species])
```

```
pmf2 = make_pmf_norm(culmen_map[species])
joint_map[species] = make_joint(pmf1, pmf2)
```

The following figure compares a scatter plot of the data to the contours of the joint distributions, assuming independence.

```
from utils import plot_contour

scatterplot(df, var1, var2)
for species in hypos:
    plot_contour(joint_map[species], alpha=0.5)
```

The contours of a joint normal distribution form ellipses. In this example, because the features are uncorrelated, the ellipses are aligned with the axes. But they are not well aligned with the data.

We can make a better model of the data, and use it to compute better likelihoods, with a multivariate normal distribution.

✓ Multivariate Normal Distribution

As we have seen, a univariate normal distribution is characterized by its mean and standard deviation.

A multivariate normal distribution is characterized by the means of the features and the **covariance matrix**, which contains **variances**, which quantify the spread of the features, and the **covariances**, which quantify the relationships among them.

We can use the data to estimate the means and covariance matrix for the population of penguins. First I'll select the columns we want.

```
features = df[[var1, var2]]
```

And compute the means.

```
mean = features.mean()
mean
```

We can also compute the covariance matrix:

```
cov = features.cov()
cov
```

The result is a `DataFrame` with one row and one column for each feature. The elements on the diagonal are the variances; the elements off the diagonal are covariances.

By themselves, variances and covariances are hard to interpret. We can use them to compute standard deviations and correlation coefficients, which are easier to interpret, but the details of that calculation are not important right now.

Instead, we'll pass the covariance matrix to `multivariate_normal`, which is a SciPy function that creates an object that represents a multivariate normal distribution.

As arguments it takes a sequence of means and a covariance matrix:

```
from scipy.stats import multivariate_normal

multinorm = multivariate_normal(mean, cov)
```

The following function makes a `multivariate_normal` object for each species.

```
def make_multinorm_map(df, colnames):
    """Make a map from each species to a multivariate normal."""
    multinorm_map = {}
    grouped = df.groupby('Species2')
    for species, group in grouped:
        features = group[colnames]
        mean = features.mean()
        cov = features.cov()
        multinorm_map[species] = multivariate_normal(mean, cov)
    return multinorm_map
```

Here's how we make this map for the first two features, flipper length and culmen length.

```
multinorm_map = make_multinorm_map(df, [var1, var2])
```

✓ Visualizing a Multivariate Normal Distribution

This section uses some NumPy magic to generate contour plots for multivariate normal distributions. If that's interesting for you, great! Otherwise, feel free to skip to the results. In the next section we'll do the actual classification, which turns out to be easier than the visualization.

I'll start by making a contour map for the distribution of features among Adélie penguins.

Here are the univariate distributions for the two features we'll use and the multivariate distribution we just computed.

```
norm1 = flipper_map['Adelie']
norm2 = culmen_map['Adelie']
multinorm = multinorm_map['Adelie']
```

I'll make a discrete `Pmf` approximation for each of the univariate distributions.

```
pmf1 = make_pmf_norm(norm1)
pmf2 = make_pmf_norm(norm2)
```

And use them to make a mesh grid that contains all pairs of values.

```
X, Y = np.meshgrid(pmf1.qs, pmf2.qs)
X.shape
```

The mesh is represented by two arrays: the first contains the quantities from `pmf1` along the `x` axis; the second contains the quantities from `pmf2` along the `y` axis.

In order to evaluate the multivariate distribution for each pair of values, we have to "stack" the arrays.

```
pos = np.dstack((X, Y))
pos.shape
```

The result is a 3-D array that you can think of as a 2-D array of pairs. When we pass this array to `multinorm.pdf`, it evaluates the probability density function of the distribution for each pair of values.

```
densities = multinorm.pdf(pos)
densities.shape
```

The result is an array of probability densities. If we put them in a `DataFrame` and normalize them, the result is a discrete approximation of the joint distribution of the two features.

```
from utils import normalize

joint = pd.DataFrame(densities, columns=pmf1.qs, index=pmf2.qs)
normalize(joint)
```

Here's what the result looks like.

```
plot_contour(joint)
decorate(xlabel=var1,
        ylabel=var2)
```

The contours of a multivariate normal distribution are still ellipses, but now that we have taken into account the correlation between the features, the ellipses are no longer aligned with the axes.

The following function encapsulate the steps we just did.

```
def make_joint(norm1, norm2, multinorm):
    """Make a joint distribution.

    norm1: `norm` object representing the distribution of the first feature
    norm2: `norm` object representing the distribution of the second feature
    multinorm: `multivariate_normal` object representing the joint distribut
    """
    pmf1 = make_pmf_norm(norm1)
    pmf2 = make_pmf_norm(norm2)
    X, Y = np.meshgrid(pmf1.qs, pmf2.qs)
    pos = np.dstack((X, Y))
    densities = multinorm.pdf(pos)
    joint = pd.DataFrame(densities, columns=pmf1.qs, index=pmf2.qs)
    return joint
```

The following figure shows a scatter plot of the data along with the contours of the multivariate normal distribution for each species.

```
scatterplot(df, var1, var2)

for species in hypos:
    norm1 = flipper_map[species]
    norm2 = culmen_map[species]
    multinorm = multinorm_map[species]
    joint = make_joint(norm1, norm2, multinorm)
    plot_contour(joint, alpha=0.5)
```

Because the multivariate normal distribution takes into account the correlations between features, it is a better model for the data. And there is less overlap in the contours of the three distributions, which suggests that they should yield better classifications.

✓ A Less Naive Classifier

In a previous section we used `update_penguin` to update a prior `Pmf` based on observed data and a collection of `norm` objects that model the distribution of observations under each hypothesis. Here it is again:

```
def update_penguin(prior, data, norm_map):
    """Update hypothetical species."""
    hypos = prior.qs
    likelihood = [norm_map[hypo].pdf(data) for hypo in hypos]
    posterior = prior * likelihood
    posterior.normalize()
    return posterior
```

Last time we used this function, the values in `norm_map` were `norm` objects, but it also works if they are `multivariate_normal` objects.

We can use it to classify a penguin with flipper length 193 and culmen length 48:

```
data = 193, 48
update_penguin(prior, data, multinorm_map)
```

A penguin with those measurements is almost certainly a Chinstrap.

Now let's see if this classifier does any better than the naive Bayesian classifier. I'll apply it to each penguin in the dataset:

```
df['Classification'] = "None"

for i, row in df.iterrows():
    data = row[colnames]
    posterior = update_penguin(prior, data, multinorm_map)
    df.loc[i, 'Classification'] = posterior.idxmax()
```

And compute the accuracy:

```
accuracy(df)
```

It turns out to be only a little better: the accuracy is 95.3%, compared to 94.7% for the naive Bayesian classifier.

Summary

In this chapter, we implemented a naive Bayesian classifier, which is "naive" in the sense that it assumes that the features it uses for classification are independent.

To see how bad that assumption is, we also implemented a classifier that uses a multivariate normal distribution to model the joint distribution of the features, which includes their dependencies.

In this example, the non-naive classifier is only marginally better. In one way, that's disappointing. After all that work, it would have been nice to see a bigger improvement. But in another way, it's good news. In general, a naive Bayesian classifier is easier to implement and requires less computation. If it works nearly as well as a more complex algorithm, it might be a good choice for practical purposes. Speaking of practical purposes, you might have noticed that this example isn't very useful. If we want to identify the species of a penguin, there are easier ways than measuring its flippers and beak.

But there *are* scientific uses for this type of classification. One of them is the subject of the research paper we started with: [sexual dimorphism](#), that is, differences in shape between male and female animals.

In some species, like angler fish, males and females look very different. In other species, like mockingbirds, they are difficult to tell apart. And dimorphism is worth studying because it provides insight into social behavior, sexual selection, and evolution.

One way to quantify the degree of sexual dimorphism in a species is to use a classification algorithm like the one in this chapter. If you can find a set of features that makes it possible to classify individuals by sex with high accuracy, that's evidence of high dimorphism.

As an exercise, you can use the dataset from this chapter to classify penguins by sex and see which of the three species is the most dimorphic.

✓ Exercises

Exercise: In my example I used culmen length and flipper length because they seemed to provide the most power to distinguish the three species. But maybe we can do better by using more features.

Make a naive Bayesian classifier that uses all four measurements in the dataset: culmen length and depth, flipper length, and body mass. Is it more accurate than the model with two features?

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: One of the reasons the penguin dataset was collected was to quantify sexual dimorphism in different penguin species, that is, physical differences between male and female penguins. One way to quantify dimorphism is to use measurements to classify penguins by sex. If a species is more dimorphic, we expect to be able to classify them more accurately.

As an exercise, pick a species and use a Bayesian classifier (naive or not) to classify the penguins by sex. Which features are most useful? What accuracy can you achieve?

Note: One Gentoo penguin has an invalid value for `Sex`. I used the following code to select one species and filter out invalid data.

```
gentoo = (df['Species2'] == 'Gentoo')
subset = df[gentoo].copy()
```

```
subset['Sex'].value_counts()
```

```
valid = df['Sex'] != '.'
valid.sum()
```

```
subset = df[valid & gentoo].copy()
```

OK, you can finish it off from here.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Solution goes here

Solution goes here

Solution goes here

Solution goes here

Solution goes here

Solution goes here

Solution goes here

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: [Attribution-NonCommercial-ShareAlike 4.0 International \(CC BY-NC-SA 4.0\)](https://creativecommons.org/licenses/by-nc-sa/4.0/)

Lesson Notes

MODULE 3 | LESSON 1

BAYESIAN STATISTICS IN CLASSIFICATION

Reading Time	60 minutes
Prior Knowledge	Basic probability theory, Financial markets
Keywords	Bayesian statistics, Likelihood, Posterior, Prior

In the previous module, we looked at unsupervised machine learning. We will now use the explanatory variables to predict a target variable. This will be the first supervised machine learning algorithm that we'll cover.

1. The Bayesian Framework

We often want to find the probability that an observation will take on a certain class value, that is a success/failure or buy/hold, as an example of a binary classification problem. These events are known as a collective exhaustive set of events, in which there are only two possible outcomes, say A and $\bar{A}$, $P(A) + P(\bar{A}) = 1$. Therefore, if we know the probability of one outcome, we can obtain the probability of the other by simply taking the complement, i.e., $P(\bar{A}) = 1 - P(A)$.

A probability of an event can depend on whether another event has taken place. For instance, we might consider the probability of a student passing an exam given that the student has studied for over 40 hours. In this case, the probability of passing the exam should be higher than if the student had studied for just four hours. If we denote the events:

- S - The event that the student passes the exam
- A - The event that the student prepares for the exam

We can write the statistical notation of $P(S|A)$ as the probability of the student passing the exam given they have prepared for it. This is known as **conditional probability** and is the building block of Bayesian statistics.

The **joint probability** is the probability of events occurring together, i.e., $P(A \cap B)$ is the probability of event A and B occurring. From the example above, $P(A \cap S)$ would be the probability that the student prepares for the exam and passes, i.e., the joint probability of A and S .

1.1 Conditional Probability

For any two events A and B , the conditional probability rule is given by, $P(A|B) = \frac{P(A \cap B)}{P(B)}$ (1.1).

Let's use the above rule in an example. A sample is taken of employees in the mining and finance sector who hold positions as either manager or director. Table 1.1 shows the sample in a contingency table. Based on the sample, suppose we wanted to know the probability of an employee being a manager given they are in the finance sector. Using (1.1), we have

$$\begin{aligned} P(\text{Management}|\text{Finance}) &= \frac{P(\text{Management} \cap \text{Finance})}{P(\text{Finance})} \\ &= \frac{700/1600}{900/1600} \\ &= 0.78 \end{aligned}$$

Table 1

	Mining	Finance	TOTAL
Manager	580	700	1,280
Director	120	200	320
TOTAL	700	900	1,600

We now have an understanding of the basic probability theory needed for the Bayesian statistics approach. We will start by looking at Bayes's theorem and the Naive Bayes classifier that is based on the theorem.

Let us look at a video that will help us put this theory of probability into practice:

04:24

[Access video transcript here](#)

1.2 Bayes Theorem

Following (1.1) we have the joint probability given by $P(A \cap B) = P(A|B)P(B)$.

Keep in mind that the event of A and B are the same as B and A ; therefore, the above can also be written as $P(A \cap B) = P(B|A) \cdot P(A)$ (1.2).

Substituting (1.2) into (1.1), we obtain $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ (1.3).

Equation (1.3) is known as Bayes theorem, which forms the basis for most algorithms using the Bayesian approach. Equation (1.3) can be extended to more than just one conditional variable. In other words, instead of just B , we can have $X_1, X_2, \dots, X_n$. We write X instead to keep it consistent with the notation of explanatory variables, and we will use Y instead of A since the target or response variable is generally written as such. We will need this notation to explain the Naive Bayes classifier methodology. Now that we've seen the Bayes theorem, let's use this to build a classifier.

1.3 Naive Bayes

Suppose we have a target variable that takes on class values. With Y taking on a specific class value, say k , equation (1.3) can be written as (1.4):

$$P(Y = k|X_1, X_2, \dots, X_n) = \frac{P(X_1|Y = k)P(X_2|Y = k) \cdots P(X_n|Y = k)P(Y = k)}{P(X_1)P(X_2) \cdots P(X_n)}$$

It is important to note that equation (1.4) is true provided the explanatory variables $X_1, X_2, \dots, X_n$ are conditionally independent of Y , i.e. $P(X_1, X_2, \dots, X_n|Y = k) = \prod_{i=1}^n P(X_i|Y = k)$. This is the "naive" assumption made for the classifier and hence the name. Naive Bayes assumes conditional independence whereas Bayes's theorem does not. The denominator can also be written as,

$$P(X_1 \cdots X_n) = \sum_j P(Y = j) \prod_{i=1}^n P(X_i|Y = j)$$

by summing over all possible classes j of the target variable. This is a property of marginalization. There are specific names given to the terms in (1.4), namely:

- $P(X_1|Y = k) \cdot P(X_2|Y = k) \cdots P(X_n|Y = k) = \textit{likelihood}$
- $P(Y = k) = \textit{Prior}$
- $P(X_1 \cdot X_2 \cdots X_n) = \textit{Evidence}$
- $P(Y = k|X_1, X_2, \dots, X_n) = \textit{Posterior}$

so we have (1.5):

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

To help you conceptualize this, please watch the following video.

03:44

[Access video transcript here](#)

1.4 Naive Bayes Example

Let us look at an example to understand the classifier. Table 1.2 contains credit risk data, specifically credit card defaulters and non-defaulters on their loans. There are two explanatory variables namely X_1 and X_2 , which are industry sector of employment and length of employment being more than 3 years or not. Determining the probability of defaulting on a loan is an important focus in the banking industry.

Table 2

	X_1	X_1	X_1	X_2	X_2	
	Mining	Finance	Agriculture	Employed > 3 yrs	Employed 3 yrs or less	TOTAL
Default (y=1)	250	600	150	300	700	1000
Non-default (y=0)	1250	6000	1750	1500	7500	9000
TOTAL	1500	6600	1900	1800	8200	10000

If we wanted the likelihood for an employee in the mining sector given they defaulted, we would use (1.1) to obtain (1.6):

$$\begin{aligned}
 P(X_1 = \text{Mining} | Y = 1) &= \frac{250}{1000} \\
 &= 0.25
 \end{aligned}$$

and similarly, the likelihood of being employed for 3 years or less given the employee defaulted is

$$\begin{aligned}
 P(X_2 = \text{Employed 3 yrs or less} | Y = 1) &= \frac{700}{1000} \\
 &= 0.7.
 \end{aligned}$$

We could now determine the Naive Bayes probability of defaulting given the employee is in the mining sector and has been employed for three years or less using (1.4). For simplicity, let M = Mining and Eu = Employed 3 years or less.

$$P(Y = 1 | X_1 = M, X_2 = Eu) = \frac{P(X_1 = M | Y = 1) \cdot P(X_2 = Eu | Y = 1) \cdot P(Y = 1)}{P(X_1 = M) \cdot P(X_2 = Eu)}$$

$$= \frac{(0.25)(0.7)(1000/10000)}{(9000/10000)(1250/9000)(7500/9000) + (1000/10000)(250/1000)(700/1000)} = 0.14.$$

Similarly, the $P(Y = 0 | X_1 = \text{Mining}, X_2 = \text{Employed 3 yrs or less}) = 0.85$. The classifier would assign the client as a non-defaulter since they have a higher probability of non-default. A useful reference for more details on Bayesian analysis can be found in chapter 12 of the Downey reading.

We have shown how the Naive Bayes classifier works and the assumptions made. The classifier can also be extended to continuous predictor variables with the assumption of normality. We've also included a [link to another helpful video](#).

To consolidate the information presented above watch the following video:



07:14

[Access video transcript here](#)

2. Conclusion

In this lesson, we presented the methodology behind Bayesian statistics and the workings of the Naive Bayes classifier. We saw this probability approach used to address classification problems by using the prior and likelihood probabilities to calculate the chance of being classified as a particular class. A simplified example with few predictors is used to grasp the concept. The following lesson will implement this tool in a practical financial problem with more predictors. Therefore, we'll incorporate Python to assist with the heavy lifting.



9 Bayesian methods

This section is dedicated to the subset of machine learning that makes prior assumptions on parameters. Before we explain how Bayes' theorem can be applied to simple building blocks in machine learning, we introduce some notations and concepts in the subsection below. Good references for Bayesian analysis are Gelman et al. (2013) and Kruschke (2014). The latter, like the present book, illustrates the concepts with many lines of R code. Bayesian inference is used in Feng and He (2021) to estimate conditional expected returns and residual covariance matrices with a view to portfolio choice.

9.1 The Bayesian framework

Up to now, the models that have been presented rely on data only. This approach is often referred to as '**frequentist**'. Given one dataset, a frequentist will extract (i.e., estimate) a unique set of optimal parameters and consider it to be the best model. Bayesians, on the other hand, consider datasets as **snapshots of reality** and, for them, parameters are thus random! Instead of estimating one value for parameters (e.g., a coefficient in a linear model), they are more ambitious and try to determine the **whole distribution** of the parameter.

In order to outline how that can be achieved, we introduce basic notations and results. The foundational concept in Bayesian analysis is the **conditional probability**. Given two random sets (or events) A and B , we define the probability of A knowing B (equivalently, the odds of having A , conditionally on having B) as

$$P[A|B] = \frac{P[A \cap B]}{P[B]},$$

that is, the probability of the intersection between the two sets divided by the probability of B . Likewise, the probability that both events occur is equal to $P[A \cap B] = P[A]P[B|A]$. Given n disjoint events A_i , $i = 1, \dots, n$ such that $\sum_{i=1}^n P(A_i) = 1$, then for any event B , the law of total probabilities is (or implies)

$$P(B) = \sum_{i=1}^n P(B \cap A_i) = \sum_{i=1}^n P(B|A_i)P(A_i).$$

Given this expression, we can formulate a general version of Bayes' theorem:

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{P(B)} = \frac{P(A_i)P(B|A_i)}{\sum_{i=1}^n P(B|A_i)P(A_i)}. \quad (9.1)$$

Endowed with this result, we can move forward to the core topic of this section, which is the estimation of some parameter θ (possibly a vector) given a dataset, which we denote with $\mathbf{y}$ thereby following the conventions from Gelman et al. (2013). This notation is suboptimal in this book nonetheless because in all other chapters, $\mathbf{y}$ stands for the label of a dataset.

In Bayesian analysis, one sophistication (compared to a frequentist approach) comes from the fact that the data is not almighty. The distribution of the parameter θ will be a mix between some **prior** distribution set by the statistician (the user, the analyst) and the empirical distribution from the data. More precisely, a simple application of Bayes' formula yields

$$p(\theta|\mathbf{y}) = \frac{p(\theta)p(\mathbf{y}|\theta)}{p(\mathbf{y})} \propto p(\theta)p(\mathbf{y}|\theta). \quad (9.2)$$

The interpretation is immediate: the distribution of θ knowing the data $\mathbf{y}$ is proportional to the distribution of θ times the distribution of $\mathbf{y}$ knowing θ . The term $p(\mathbf{y})$ is often omitted because it is simply a scaling number that ensures that the density sums or integrates to one.

We use a slightly different notation between Equation (9.1) and Equation (9.2). In the former, P denotes a true probability, i.e., it is a number. In the latter, p stands for the whole probability density function of θ or $\mathbf{y}$.

The whole purpose of Bayesian analysis is to compute the so-called **posterior** distribution $p(\theta|\mathbf{y})$ via the **prior** distribution $p(\theta)$ and the **likelihood function** $p(\mathbf{y}|\theta)$. Priors are sometimes qualified as informative, weakly informative or uninformative, depending on the degree to which the user is confident on the relevance and robustness of the prior. The simplest way to define a non-informative prior is to set a constant (uniform) distribution over some realistic interval(s).

The most challenging part is usually the likelihood function. The easiest way to solve the problem is to resort to a specific distribution (possibly a parametric family) for the distribution of the data and then consider that observations are i.i.d., just as in a simple maximum likelihood inference. If we assume that new parameters for the distributions are gathered into $\boldsymbol{\lambda}$, then the likelihood can be written as

$$p(\mathbf{y}|\boldsymbol{\theta}, \boldsymbol{\lambda}) = \prod_{i=1}^I f_{\lambda}(y_i; \boldsymbol{\beta}), \quad (9.3)$$

but in this case the problem becomes slightly more complex because adding new parameters changes the posterior distribution to $p(\boldsymbol{\theta}, \boldsymbol{\lambda}|\mathbf{y})$. The user must find out the joint distribution of $\boldsymbol{\theta}$ and $\boldsymbol{\lambda}$ - given $\mathbf{y}$. Because of their nested structure, these models are often called **hierarchical models**.

Bayesian methods are widely used for portfolio choice. The rationale is that the distribution of asset returns depends on some parameter and the main issue is to determine the posterior distribution. We very briefly review a vast literature below. Bayesian asset allocation is investigated in Lai et al. (2011) (via stochastic optimization), Guidolin and Liu (2016) and Dangl and Weissensteiner (2020). Shrinkage techniques (of means and covariance matrices) are tested in Frost and Savarino (1986), Kan and Zhou (2007) and DeMiguel, Martín-Utrera, and Nogales (2015). In a similar vein, Tu and Zhou (2010) build priors that are coherent with asset pricing theories. Finally, Bauder et al. (2020) sample portfolio returns which allows to derive a Bayesian optimal frontier. We invite the interested reader to also dwell in the references that are cited within these few articles.

9.2 Bayesian sampling

9.2.1 Gibbs sampling

One adjacent field of applications of Bayes' theorem is **simulation**. Suppose we want to simulate the multivariate distribution of a random vector $\mathbf{X}$ given by its density $p = p(x_1, \dots, x_J)$. Often, the full distribution is complex, but its marginals are more accessible. Indeed, they are simpler because they depend on only one variable (when all other values are known):

$$p(X_j = x_j | X_1 = x_1, \dots, X_{j-1} = x_{j-1}, X_{j+1} = x_{j+1}, \dots, X_J = x_J) = p(X_j = x_j | \mathbf{X}_{-j} = \mathbf{x}_{-j}),$$

where we use the compact notation $\mathbf{X}_{-j}$ for all variables except X_j . One way to generate samples with law p is the following and relies both on the knowledge of the conditionals $p(x_j | \mathbf{x}_{-j})$ and on the notion of **Markov Chain Monte Carlo**, which we outline below. The process is iterative and assumes that it is possible to draw samples of the aforementioned conditionals. We write x_j^m for the m^{th} sample of the j^{th} variable (X_j). The simulation starts with a prior (or fixed, or random) sample $\mathbf{x}^0 = (x_1^0, \dots, x_J^0)$. Then, for a sufficiently large number of times, say T , new samples are drawn according to

$$\begin{aligned} x_1^{m+1} &= p(X_1 | X_2 = x_2^m, \dots, X_J = x_J^m); \\ x_2^{m+1} &= p(X_2 | X_1 = x_1^{m+1}, X_3 = x_3^m, \dots, X_J = x_J^m); \\ &\dots \\ x_J^{m+1} &= p(X_J | X_1 = x_1^{m+1}, X_2 = x_2^{m+1}, \dots, X_{J-1} = x_{J-1}^{m+1}). \end{aligned}$$

The important detail is that after each line, the value of the variable is updated. Hence, in the second line, X_2 is sampled with the knowledge of $X_1 = x_1^{m+1}$ and in the last line, all variables except X_J have been updated to their $(m+1)^{th}$ state. The above algorithm is called Gibbs sampling. It relates to Markov chains because each new iteration depends only on the previous one.

Under some technical assumptions, as T increases, the distribution of $\mathbf{x}_T$ converges to that of p . The conditions under which the convergence occurs have been widely discussed in series of articles in the 1990s. The interested reader can have a look for instance at Tierney (1994), Roberts and Smith (1994), as well as at section 11.7 of Gelman et al. (2013).

Sometimes, the full distribution is complex and the conditional laws are hard to determine and to sample. Then, a more general method, called Metropolis-Hastings, can be used that relies on the rejection method for the simulation of random variables.

9.2.2 Metropolis-Hastings sampling

The Gibbs algorithm can be considered as a particular case of the Metropolis-Hastings (MH) method, which, in its simplest version, was introduced in Metropolis and Ulam (1949). The premise is similar: the aim is to simulate random variables that follow $p(\mathbf{x})$ with the ability to sample from a simpler form $p(\mathbf{x}|\mathbf{y})$ which gives the probability of the future state $\mathbf{x}$, given the past one $\mathbf{y}$.

Once an initial value for $\mathbf{x}$ has been sampled ($\mathbf{x}_0$), each new iteration (m) of the simulation takes place in three stages:

1. generate a candidate value $\mathbf{x}'_{m+1}$ from $p(\mathbf{x}|\mathbf{x}_m)$,
2. compute the acceptance ratio $\alpha = \min \left(\frac{p(\mathbf{x}'_{m+1})p(\mathbf{x}_m|\mathbf{x}'_{m+1})}{p(\mathbf{x}_m)p(\mathbf{x}'_{m+1}|\mathbf{x}_m)} \right)$

3. pick $\mathbf{x}_{m+1} = \mathbf{x}'_{m+1}$ with probability α or stick with the previous value ($\mathbf{x}_{m+1} = \mathbf{x}_m$) with probability $1 - \alpha$.

The interpretation of the acceptance ratio is not straightforward in the general case. When the sampling generator is symmetric ($p(\mathbf{x}|\mathbf{y}) = p(\mathbf{y}|\mathbf{x})$), the candidate is always chosen whenever $p(\mathbf{x}'_{m+1}) \geq p(\mathbf{x}_m)$. If the reverse condition holds ($p(\mathbf{x}'_{m+1}) < p(\mathbf{x}_m)$), then the candidate is retained with odds equal to $p(\mathbf{x}'_{m+1})/p(\mathbf{x}_m)$, which is the ratio of likelihoods. The more likely the new proposal, the higher the odds of retaining it.

Often, the first simulations are discarded in order to leave time to the chain to converge to a high probability region. This procedure (often called '*burn in*') ensures that the first retained samples are located in a zone that is likely, i.e., that they are more representative of the law we are trying to simulate.

For the sake of brevity, we stick to a succinct presentation here, but some additional details are outlined in section 11.2 of Gelman et al. (2013) and in chapter 7 of Kruschke (2014).

9.3 Bayesian linear regression

Because Bayesian concepts are rather abstract, it is useful to illustrate the theoretical notions with a simple example. In a linear model, $y_i = \mathbf{x}_i \mathbf{b} + \epsilon_i$ and it is often statistically assumed that the ϵ_i are i.i.d. and normally distributed with zero mean and variance σ^2 . Hence, the likelihood of Equation (9.3) translates into

$$p(\boldsymbol{\epsilon}|\mathbf{b}, \sigma) = \prod_{i=1}^I \frac{e^{-\frac{\epsilon_i^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}} = (\sigma\sqrt{2\pi})^{-I} e^{-\sum_{i=1}^I \frac{\epsilon_i^2}{2\sigma^2}}.$$

In a regression analysis, the data is given both by $\mathbf{y}$ and by $\mathbf{X}$, hence both are reported in the notations. Simply acknowledging that $\boldsymbol{\epsilon} = \mathbf{y} - \mathbf{X}\mathbf{b}$, we get

$$\begin{aligned} p(\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma) &= \prod_{i=1}^I \frac{e^{-\frac{\epsilon_i^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}} \\ &= (\sigma\sqrt{2\pi})^{-I} e^{-\sum_{i=1}^I \frac{(y_i - \mathbf{x}_i' \mathbf{b})^2}{2\sigma^2}} = (\sigma\sqrt{2\pi})^{-I} e^{-\frac{(\mathbf{y} - \mathbf{X}\mathbf{b})'(\mathbf{y} - \mathbf{X}\mathbf{b})}{2\sigma^2}} \\ &= \underbrace{(\sigma\sqrt{2\pi})^{-I} e^{-\frac{(\mathbf{y} - \mathbf{X}\mathbf{b})'(\mathbf{y} - \mathbf{X}\mathbf{b})}{2\sigma^2}}}_{\text{depends on } \sigma, \text{ not } \mathbf{b}} \times \underbrace{e^{-\frac{(\mathbf{b} - \hat{\mathbf{b}})' \mathbf{X}' \mathbf{X} (\mathbf{b} - \hat{\mathbf{b}})}{2\sigma^2}}}_{\text{depends on both } \sigma, \text{ and } \mathbf{b}}. \end{aligned} \quad (9.4)$$

In the last line, the second term is a function of the difference $\mathbf{b} - \hat{\mathbf{b}}$, where $\hat{\mathbf{b}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$. This is not surprising: $\hat{\mathbf{b}}$ is a natural benchmark for the mean of $\mathbf{b}$. Moreover, introducing $\hat{\mathbf{b}}$ yields a relatively simple form for the probability.

The above expression is the frequentist (data-based) block of the posterior: the likelihood. If we want to obtain a tractable expression for the posterior, we need to find a prior component that has a form that will combine well with this likelihood. These forms are called **conjugate priors**. A natural candidate for the right part (that depends on both $\mathbf{b}$ and σ) is the multivariate Gaussian density:

$$p[\mathbf{b}|\sigma] = \sigma^{-k} e^{-\frac{(\mathbf{b} - \mathbf{b}_0)' \boldsymbol{\Lambda}_0 (\mathbf{b} - \mathbf{b}_0)}{2\sigma^2}}, \quad (9.5)$$

where we are obliged to condition with respect to σ . The density has prior mean $\mathbf{b}_0$ and prior covariance matrix $\boldsymbol{\Lambda}_0^{-1}$. This prior gets us one step closer to the posterior because

$$\begin{aligned} p[\mathbf{b}, \sigma|\mathbf{y}, \mathbf{X}] &\propto p[\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma] p[\mathbf{b}, \sigma] \\ &\propto p[\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma] p[\mathbf{b}|\sigma] p[\sigma]. \end{aligned} \quad (9.6)$$

In order to fully specify the cascade of probabilities, we need to take care of σ and set a density of the form

$$p[\sigma^2] \propto (\sigma^2)^{-1-a_0} e^{-\frac{b_0}{2\sigma^2}}, \quad (9.7)$$

which is close to that of the left part of (9.4). This corresponds to an inverse gamma distribution for the variance with prior parameters a_0 and b_0 (this scalar notation is not optimal because it can be confused with the prior mean $\mathbf{b}_0$ so we must pay extra attention).

Now, we can simplify $p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}]$ with (9.4), (9.5) and (9.7):

$$p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}] \propto (\sigma \sqrt{2\pi})^{-I} \sigma^{-2(1+a_0)} e^{-\frac{(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})'(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})}{2\sigma^2}} \\ \times e^{-\frac{(\mathbf{b}-\hat{\mathbf{b}})'\mathbf{X}'\mathbf{X}(\mathbf{b}-\hat{\mathbf{b}})}{2\sigma^2}} \sigma^{-k} e^{-\frac{(\mathbf{b}-\mathbf{b}_0)'\mathbf{\Lambda}_0(\mathbf{b}-\mathbf{b}_0)}{2\sigma^2}} e^{-\frac{b_0}{2\sigma^2}}$$

which can be rewritten

$$p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}] \propto \sigma^{-I-k-2(1+a_0)} \\ \times \exp\left(-\frac{(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})'(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}}) + (\mathbf{b}-\hat{\mathbf{b}})'\mathbf{X}'\mathbf{X}(\mathbf{b}-\hat{\mathbf{b}}) + (\mathbf{b}-\mathbf{b}_0)'\mathbf{\Lambda}_0(\mathbf{b}-\mathbf{b}_0) + b_0}{2\sigma^2}\right).$$

The above expression is simply a quadratic form in $\mathbf{b}$ and it can be rewritten after burdensome algebra in a much more compact manner:

$$p(\mathbf{b} | \mathbf{y}, \mathbf{X}, \sigma) \propto \left[\sigma^{-k} e^{-\frac{(\mathbf{b}-\mathbf{b}_*)'\mathbf{\Lambda}_*(\mathbf{b}-\mathbf{b}_*)}{2\sigma^2}} \right] \times \left[(\sigma^2)^{-1-a_*} e^{-\frac{b_*}{2\sigma^2}} \right],$$

where

$$\mathbf{\Lambda}_* = \mathbf{X}'\mathbf{X} + \mathbf{\Lambda}_0 \\ \mathbf{b}_* = \mathbf{\Lambda}_*^{-1}(\mathbf{\Lambda}_0\mathbf{b}_0 + \mathbf{X}'\mathbf{X}\hat{\mathbf{b}}) \\ a_* = a_0 + I/2 \\ b_* = b_0 + \frac{1}{2}(\mathbf{y}'\mathbf{y} + \mathbf{b}_0'\mathbf{\Lambda}_0\mathbf{b}_0 + \mathbf{b}_*'\mathbf{\Lambda}_*\mathbf{b}_*).$$

This expression has two parts: the Gaussian component which relates mostly to $\mathbf{b}$, and the inverse gamma component, entirely dedicated to σ . The mix between the prior and the data is clear. The posterior covariance matrix of the Gaussian part ($\mathbf{\Lambda}_*$) is the sum between the prior and a quadratic form from the data. The posterior mean $\mathbf{b}_*$ is a weighted average of the prior $\mathbf{b}_0$ and the sample estimator $\hat{\mathbf{b}}$. Such blends of quantities estimated from data and a user-supplied version are often called **shrinkages**. For instance, the original matrix of cross-terms $\mathbf{X}'\mathbf{X}$ is shrunk towards the prior $\mathbf{\Lambda}_0$. This can be viewed as a **regularization** procedure: the pure fit originating from the data is mixed with some ‘external’ ingredient to give some structure to the final estimation.

The interested reader can also have a look at section 16.3 of Greene (2018) (the case of conjugate priors is treated in subsection 16.3.2).

The formulae above can be long and risky to implement. Luckily, there is an R package (*spBayes*) that performs Bayesian inference for linear regression using the conjugate priors. Below, we provide one example of how it works. To simplify the code and curtail computation times, we consider two predictors: market capitalization (size anomaly) and price-to-book ratio (value anomaly). In statistics, the **precision matrix** is the inverse of the covariance matrix. In the parameters, the first two priors relate to the Gaussian law and the last two to the inverse gamma distribution:

$$f_{\text{invgamma}}(x, \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{-1-\alpha} e^{-\frac{\beta}{x}},$$

where α is the shape and β is the scale.

```
prior_mean <- c(0.01,0.1,0.1) # Average value of parameters (prior)
precision_mat <- diag(prior_mean^2) %>% solve() # Inverse cov matrix of parameters (prior)
fit_lmBayes <- bayesLMConjugate(
  R1M_Usd ~ Mkt_Cap_3M_Usd + Pb, # Model: size and value
  data = testing_sample, # Data source, here, the test sample
  n.samples = 2000, # Number of samples used
  beta.prior.mean = prior_mean, # Avg prior: size & value rewarded & unit beta
  beta.prior.precision = precision_mat, # Precision matrix
  prior.shape = 0.5, # Shape for prior distribution of sigma
  prior.rate = 0.5) # Scale for prior distribution of sigma
```

In the above specification, we must also provide a prior for the constant. By default, we set its average value to 0.01, which corresponds to a 1% average monthly return. Once the model has been estimated, we can plot the distribution of coefficient estimates.

```
fit_lmBayes$p.beta.tauSq.samples[,1:3] %>% as_tibble() %>%
  `colnames<-`(c("Intercept", "Size", "Value")) %>%
  gather(key = coefficient, value = value) %>%
  ggplot(aes(x = value, fill = coefficient)) + geom_histogram(alpha = 0.5)
```

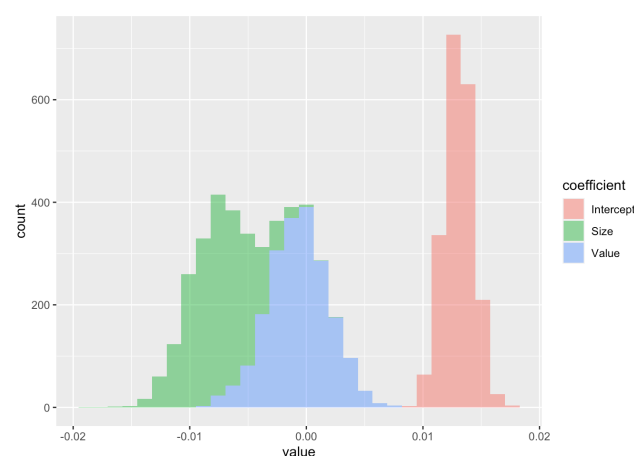


FIGURE 9.1: Distribution of linear regression coefficients (betas).

The distribution of the constant in Figure 9.1 is firmly to the right with a small dispersion, hence it is solidly positive. For the size coefficient, it is the opposite; it is negative (small firms are more profitable). With regard to value, it is hard to conclude, the distribution is balanced around zero: there is no clear exposition to the price-to-book ratio variable.

9.4 Naive Bayes classifier

Bayes' theorem can also be easily applied to **classification**. We formulate it with respect to the label and features and write

$$P[\mathbf{y}|\mathbf{X}] = \frac{P[\mathbf{X}|\mathbf{y}]P[\mathbf{y}]}{P[\mathbf{X}]} \propto P[\mathbf{X}|\mathbf{y}]P[\mathbf{y}], \quad (9.8)$$

and then split the input matrix into its column vectors $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_K)$. This yields

$$P[\mathbf{y}|\mathbf{x}_1, \dots, \mathbf{x}_K] \propto P[\mathbf{x}_1, \dots, \mathbf{x}_K|\mathbf{y}]P[\mathbf{y}]. \quad (9.9)$$

The 'naive' qualification of the method comes from a simplifying assumption on the features.¹⁹ If they are all mutually independent, then the likelihood in the above expression can be expanded into

$$P[\mathbf{y}|\mathbf{x}_1, \dots, \mathbf{x}_K] \propto P[\mathbf{y}] \prod_{k=1}^K P[\mathbf{x}_k|\mathbf{y}]. \quad (9.10)$$

The next step is to be more specific about the likelihood. This can be done non-parametrically (via kernel estimation) or with common distributions (Gaussian for continuous data, Bernoulli for binary data). In factor investing, the features are continuous, thus the Gaussian law is more adequate:

$$P[x_{i,k} = z | y_i = c] = \frac{e^{-\frac{(z-m_c)^2}{2\sigma_c^2}}}{\sigma_c \sqrt{2\pi}},$$

where c is the value of the classes taken by y and σ_c and m_c are the standard error and mean of $x_{i,k}$, conditional on y_i being equal to c . In practice, each class is spanned, the training set is filtered accordingly and σ_c and m_c are taken to be the sample statistics. This Gaussian parametrization is probably ill-suited to our dataset because the features are uniformly distributed. Even after conditioning, it is unlikely that the distribution will be even remotely close to Gaussian. Technically, this can be overcome via a double transformation method. Given a vector of features $\mathbf{x}_k$ with empirical cdf $F_{\mathbf{x}_k}$, the variable

$$\tilde{\mathbf{x}}_k = \Phi^{-1}(F_{\mathbf{x}_k}(\mathbf{x}_k)), \quad (9.11)$$

will have a standard normal law whenever $F_{\mathbf{x}_k}$ is not pathological. Non-pathological cases are when the cdf is continuous and strictly increasing and when observations lie in the open interval (0,1). If all features are independent, the transformation should not have any impact on the correlation structure. Otherwise, we refer to the literature on the NORmal-To-Anything (NORTA) method (see, e.g., H. Chen (2001) and Coqueret (2017)).

Lastly, the prior $P[\mathbf{y}]$ in Equation (9.10) is often either taken to be uniform across the classes ($1/K$ for all k) or equal to the sample distribution.

We illustrate the naive Bayes classification tool with a simple example. While the package *e1071* embeds such a classifier, the *naivebayes* library offers more options (Gaussian, Bernoulli, multinomial and nonparametric likelihoods). Below, since the features are uniformly distributed, thus the transformation in (9.11) amounts to apply the Gaussian quantile function (inverse cdf).

For visual clarity, we only use the small set of features.

```
library(naivebayes) # Load package
gauss_features_train <- training_sample %>% # Build sample
  dplyr::select(features_short) %>%
  as.matrix() %>%
  `*`(0.999) %>% # Features smaller than 1
  + (0.0001) %>% # Features larger than 0
  qnorm() %>% # Inverse Gaussian cdf
  `colnames<-`(features_short)
fit_NB_gauss <- naive_bayes(x = gauss_features_train, # Transformed features
  y = training_sample$R1M_Usd_C) # Label
layout(matrix(c(1,1,2,3,4,5,6,7), 4, 2, byrow = TRUE), # Organize graphs
  widths=c(0.9,0.45))
par(mar=c(1, 1, 1, 1))
plot(fit_NB_gauss, prob = "conditional")
```

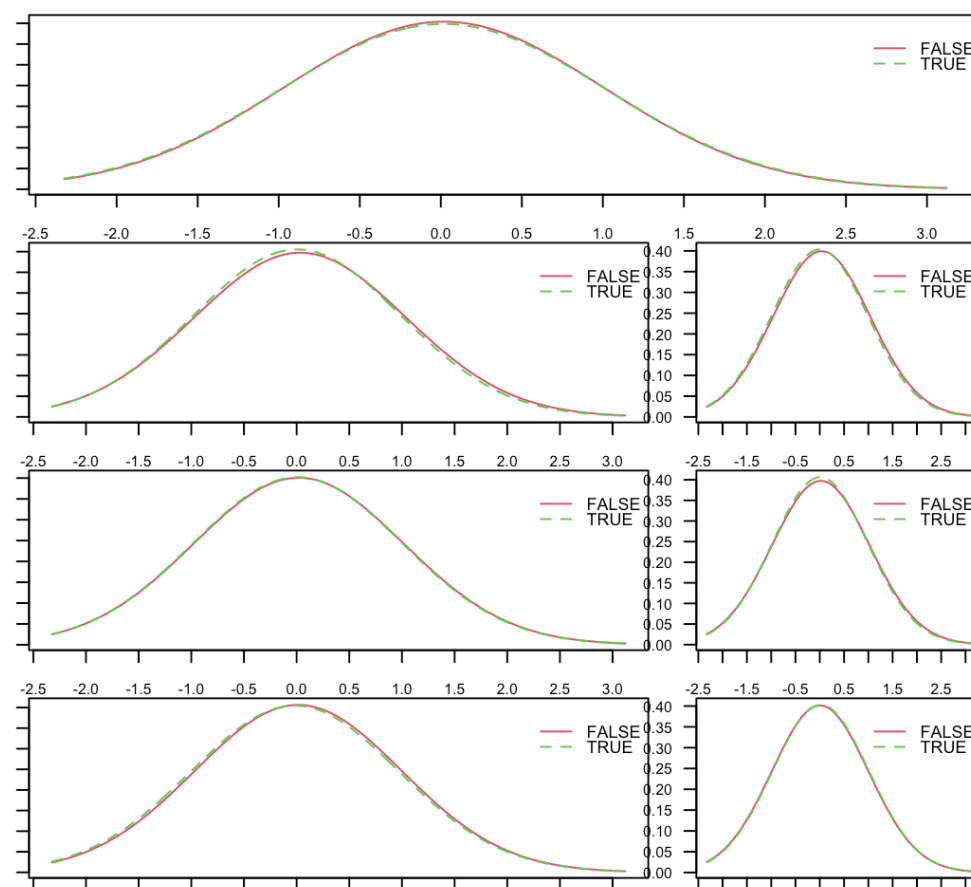


FIGURE 9.2: Distributions of predictor variables, conditional on the class of the label. TRUE is when the instance corresponds to an above median return and FALSE to a below median return.

The plots in Figure 9.2 show the distributions of the features, conditionally on each value of the label. Essentially, those are the densities $P[\mathbf{x}_k|\mathbf{y}]$. For each feature, both distributions are very similar.

As usual, once the model has been trained, the accuracy of predictions can be evaluated.

```
gauss_features_test <- testing_sample %>%
  dplyr::select(features_short) %>%
  as.matrix() %>%
  `*`(0.999) %>%
  + (0.0001) %>%
  qnorm() %>%
  `colnames<-`(features_short)
mean(predict(fit_NB_gauss, gauss_features_test) == testing_sample$R1M_Usd_C) # Hit ratio
```

The performance of the classifier is not satisfactory as it underperforms a random guess.

9.5 Bayesian additive trees

9.5.1 General formulation

Bayesian additive regression trees (BARTs) are an ensemble technique that mixes Bayesian thinking and regression trees. In spirit, they are close to the tree ensembles seen in Chapter 6, but they differ greatly in their implementation. In BARTs like in Bayesian regressions, the regularization comes from the prior. The original article is Chipman, George, and McCulloch (2010) and the implementation (in R) follows Sparapani, Spanbauer, and McCulloch (2019). BARTs have been used in Shu and Tiwari (2021) to identify which characteristics are priced. The authors report that stocks' market capitalization is the only one that matters.

Formally, the model is an aggregation of M models, which we write as

$$y = \sum_{m=1}^M \mathcal{T}_m(q_m, \mathbf{w}_m, \mathbf{x}) + \epsilon, \quad (9.12)$$

where ϵ is a Gaussian noise with variance σ^2 , and the $\mathcal{T}_m = \mathcal{T}_m(q_m, \mathbf{w}_m, \mathbf{x})$ are decision trees with structure q_m and weights vectors $\mathbf{w}_m$. This decomposition of the tree is the one we used for boosted trees and is illustrated in Figure 6.5. q_m codes all splits (variables chosen for the splits and levels of the splits) and the vectors $\mathbf{w}_m$ correspond to the leaf values (at the terminal nodes).

At the macro-level, BARTs can be viewed as traditional Bayesian objects, where the parameters θ are all of the unknowns coded through q_m , $\mathbf{w}_m$ and σ^2 and where the focus is set on determining the posterior

$$(q_m, \mathbf{w}_m, \sigma^2) | (\mathbf{X}, \mathbf{Y}). \quad (9.13)$$

Given particular forms of priors for $(q_m, \mathbf{w}_m, \sigma^2)$, the algorithm draws the parameters using a combination of Metropolis-Hastings and Gibbs samplers.

9.5.2 Priors

The definition of priors in tree models is delicate and intricate. The first important assumption is independence: independence between σ^2 and all other parameters and independence between trees, that is, between couples $(q_m, \mathbf{w}_m)$ and $(q_n, \mathbf{w}_n)$ for $m \neq n$. This assumption makes BARTs closer to random forests in spirit and further from boosted trees. This independence entails

$$P((q_1, \mathbf{w}_1), \dots, (q_M, \mathbf{w}_M), \sigma^2) = P(\sigma^2) \prod_{m=1}^M P(q_m, \mathbf{w}_m).$$

Moreover, it is customary (for simplicity) to separate the structure of the tree (q_m) and the terminal weights ($\mathbf{w}_m$), so that by a Bayesian conditioning

$$P((q_1, \mathbf{w}_1), \dots, (q_M, \mathbf{w}_M), \sigma^2) = \underbrace{P(\sigma^2)}_{\text{noise term}} \prod_{m=1}^M \underbrace{P(\mathbf{w}_m | q_m)}_{\text{tree weights}} \underbrace{P(q_m)}_{\text{tree struct.}} \quad (9.14)$$

It remains to formulate the assumptions for each of the three parts.

We start with the trees' structures, q_m . Trees are defined by their splits (at nodes) and these splits are characterized by the splitting variable and the splitting level. First, the size of trees is parametrized such that a node at depth d is nonterminal with probability given by

$$\alpha(1 + d)^{-\beta}, \quad \alpha \in (0, 1), \quad \beta > 0. \quad (9.15)$$

The authors recommend to set $\alpha = 0.95$ and $\beta = 2$. This gives a probability of 5% to have 1 node, 55% to have 2 nodes, 28% to have 3 nodes, 9% to have 4 nodes and 3% to have 5 nodes. Thus, the aim is to force relatively shallow structures.

Second, the choice of splitting variables is driven by a generalized Bernoulli (categorical) distribution which defines the odds of picking one particular feature. In the original paper by Chipman, George, and McCulloch (2010), the vector of probabilities was uniform (each predictor has the same odds of being chosen for the split). This vector can also be random and sampled from a more flexible Dirichlet distribution. The level of the split is drawn uniformly on the set of possible values for the chosen predictor.

Having determined the prior of structure of the tree q_m , it remains to fix the terminal values at the leaves ($\mathbf{w}_m|q_m$). The weights at all leaves are assumed to follow a Gaussian distribution $\mathcal{N}(\mu_\mu, \sigma_\mu^2)$, where $\mu_\mu = (y_{\min} + y_{\max})/2$ is the center of the range of the label values. The variance σ_μ^2 is chosen such that μ_μ plus or minus two times σ_μ^2 covers 95% of the range observed in the training dataset. Those are default values and can be altered by the user.

Lastly, for computational purposes similar to those of linear regressions, the parameter σ^2 (the variance of ϵ in (9.12)) is assumed to follow an inverse Gamma law $\text{IG}(\nu/2, \lambda\nu/2)$ akin to that used in Bayesian regressions. The parameters are by default computed from the data so that the distribution of σ^2 is realistic and prevents overfitting. We refer to the original article, section 2.2.4, for more details on this topic.

In sum, in addition to M (number of trees), the prior depends on a small number of parameters: α and β (for the tree structure), μ_μ and σ_μ^2 (for the tree weights) and ν and λ (for the noise term).

9.5.3 Sampling and predictions

The posterior distribution in (9.13) cannot be obtained analytically but simulations are an efficient shortcut to the model (9.12). Just as in Gibbs and Metropolis-Hastings sampling, the distribution of simulations is expected to converge to the sought posterior. After some burn-in sample, a prediction for a newly observed set $\mathbf{x}_*$ will simply be the average (or median) of the predictions from the simulations. If we assume S simulations after burn-in, then the average is equal to

$$\tilde{y}(\mathbf{x}_*) := \frac{1}{S} \sum_{s=1}^S \sum_{m=1}^M \mathcal{T}_m \left(q_m^{(s)}, \mathbf{w}_m^{(s)}, \mathbf{x}_* \right).$$

The complex part is naturally to generate the simulations. Each tree is sampled using the Metropolis-Hastings method: a tree is proposed, but it replaces the existing one only under some (possibly random) criterion. This procedure is then repeated in a Gibbs-like fashion.

Let us start with the MH building block. We seek to simulate the conditional distribution

$$(q_m, \mathbf{w}_m) \mid (q_{-m}, \mathbf{w}_{-m}, \sigma^2, \mathbf{y}, \mathbf{x}),$$

where q_{-m} and $\mathbf{w}_{-m}$ collect the structures and weights of all trees except for tree number m . One tour de force in BART is to simplify the above Gibbs draws to

$$(q_m, \mathbf{w}_m) \mid (\mathbf{R}_m, \sigma^2),$$

where $\mathbf{R}_m = \mathbf{y} - \sum_{l \neq m} \mathcal{T}_l(q_l, \mathbf{w}_l, \mathbf{x})$ is the partial residual on a prediction that excludes the m^{th} tree.

The new MH proposition for q_m is based on the previous tree and there are three possible (and random) alterations to the tree:

- growing a terminal node (increase the complexity of the tree by adding a supplementary leaf);
- pruning a pair of terminal nodes (the opposite operation: reducing complexity);
- changing splitting rules.

For simplicity, the third option is often excluded. Once the tree structure is defined (i.e., sampled), the terminal weights are independently drawn according to a Gaussian distribution $\mathcal{N}(\mu_\mu, \sigma_\mu^2)$.

After the tree is sampled, the MH principle requires that it be accepted or rejected based on some probability. This probability increases with the odds that the new tree increases the likelihood of the model. Its detailed computation is cumbersome and we refer to section 2.2 in Sparapani, Spanbauer, and McCulloch (2019) for details on the matter.

Now, we must outline the overarching Gibbs procedure. First, the algorithm starts with trees that are simple nodes. Then, a specified number of loops include the following *sequential* steps:

Step	Task
1	sample $(q_1, \mathbf{w}_1) \mid (\mathbf{R}_1, \sigma^2)$;
2	sample $(q_2, \mathbf{w}_2) \mid (\mathbf{R}_2, \sigma^2)$;
...	...;
m	sample $(q_m, \mathbf{w}_m) \mid (\mathbf{R}_m, \sigma^2)$;
...	...;
M	sample $(q_M, \mathbf{w}_M) \mid (\mathbf{R}_M, \sigma^2)$; (last tree)

Step	Task
M+1	sample σ^2 given the full residual $\mathbf{R} = \mathbf{y} - \sum_{l=1}^M \mathcal{T}_l(q_l, \mathbf{w}_l, \mathbf{x})$

At each step m , the residual $\mathbf{R}_m$ is updated with the values from step $m - 1$. We illustrate this process in Figure 9.3 in which $M = 3$. At step 1, a partition is proposed for the first tree, which is a simple node. In this particular case, the tree is accepted. In this scheme, the terminal weights are omitted for simplicity. At step 2, another partition is proposed for the tree, but it is rejected. In the third step, the proposition for the third is accepted. After the third step, a new value for σ^2 is drawn and a new round of Gibbs sampling can commence.

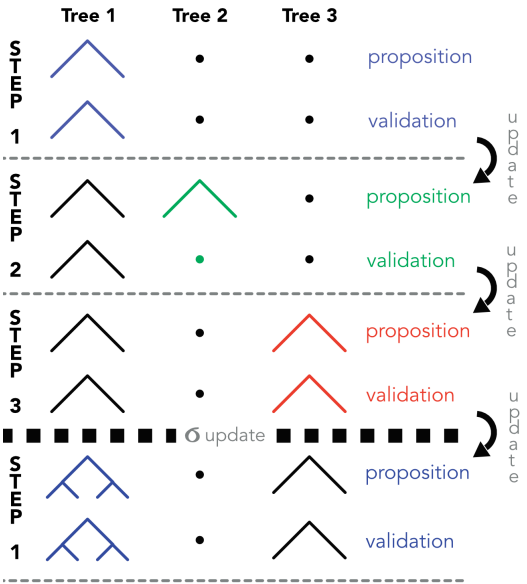


FIGURE 9.3: Diagram of the MH/Gibbs sampling of BARTs. At step 2, the proposed tree is not validated.

9.5.4 Code

There are several R packages that implement BART methods: *BART*, *bartMachine* and an older one (the original), *BayesTree*. The first one is highly efficient, hence we work with it. We resort to only a few parameters, like the power and base, which are the β and α defined in (9.15). The program is a bit verbose and delivers a few parametric details.

```

library(BART)
fit_bart <- gbart(
  x.train = dplyr::select(training_sample, features_short) %>%
    data.frame(),
  y.train = dplyr::select(training_sample, R1M_Usd) %>%
    as.matrix() ,
  x.test = dplyr::select(testing_sample, features_short) %>%
    data.frame(),
  type = "wbart",
  ntree = 20,
  nskip = 100,
  ndpost = 200,
  power = 2,
  base = 0.95)

```

Load package
 # Main function
 # Training features
 # Training label
 # Testing features
 # Option: label is continuous
 # Number of trees in the model
 # Size of burn-in sample
 # Number of posteriors drawn
 # beta in the tree structure prior
 # alpha in the tree structure prior

```
## *****Calling gbart: type=1
## *****Data:
## data:n,p,np: 198128, 7, 70208
## y1,yn: -0.049921, 0.024079
## x1,x[n*p]: 0.010000, 0.810000
## xp1,xp[np*p]: 0.270000, 0.880000
## *****Number of Trees: 20
## *****Number of Cut Points: 100 ... 100
## *****burn,nd,thin: 100,200,1
## *****Prior:beta,alpha,tau,nu,lambda,offset: 2,0.95,1.57391,3,2.84908e-31,0.0139209
## *****sigma: 0.000000
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,7,0
## *****printevery: 100
##
## MCMC
## done 0 (out of 300)
## done 100 (out of 300)
## done 200 (out of 300)
## time: 31s
## trcnt,tecnt: 200,200
```

Once the model is trained,²⁰ we evaluated its performance. We simply compute the hit ratio. The predictions are embedded within the fit variable, under the name ‘*yhat.test*’.

```
mean(fit_bart$yhat.test * testing_sample$R1M_Usd > 0)
```

```
## [1] 0.5432794
```

The performance *seems* reasonable but is by no means impressive. The data from all sampled trees is available in the *fit_bart* variable. It has nonetheless a complex structure (as is often the case with trees). The simplest information we can extract is the value of σ across all 300 simulations (see Figure 9.4).

```
data.frame(simulation = 1:300, sigma = fit_bart$sigma) %>%
  ggplot(aes(x = simulation, y = sigma)) + geom_point(size = 0.7)
```

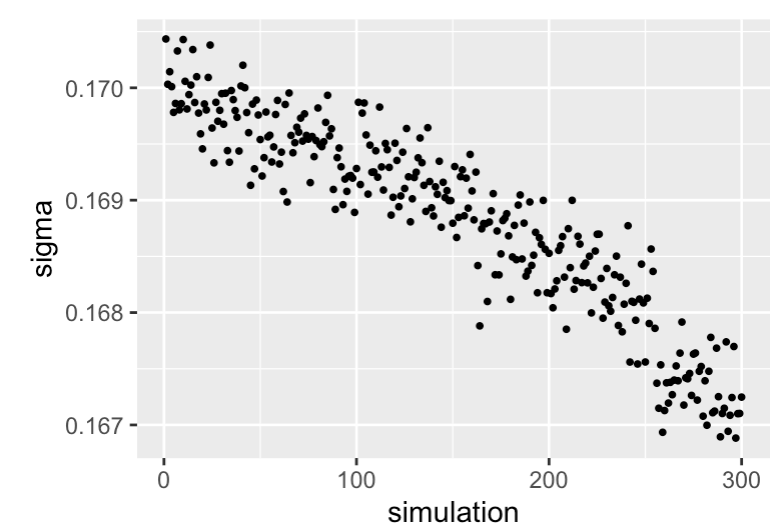


FIGURE 9.4: Evolution of sigma across BART simulations.

And we see that, as the number of samples increases, σ decreases.

```
.container-fluid main { max-width: 60rem; }
```




6

Linear Model Selection and Regularization

In the regression setting, the standard linear model

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon \quad (6.1)$$

is commonly used to describe the relationship between a response Y and a set of variables $X_1, X_2, \dots, X_p$. We have seen in Chapter 3 that one typically fits this model using least squares.

In the chapters that follow, we consider some approaches for extending the linear model framework. In Chapter 7 we generalize (6.1) in order to accommodate non-linear, but still additive, relationships, while in Chapters 8 and 10 we consider even more general non-linear models. However, the linear model has distinct advantages in terms of inference and, on real-world problems, is often surprisingly competitive in relation to non-linear methods. Hence, before moving to the non-linear world, we discuss in this chapter some ways in which the simple linear model can be improved, by replacing plain least squares fitting with some alternative fitting procedures.

Why might we want to use another fitting procedure instead of least squares? As we will see, alternative fitting procedures can yield better *prediction accuracy* and *model interpretability*.

- *Prediction Accuracy*: Provided that the true relationship between the response and the predictors is approximately linear, the least squares estimates will have low bias. If $n \gg p$ —that is, if n , the number of observations, is much larger than p , the number of variables—then the least squares estimates tend to also have low variance, and hence will perform well on test observations. However, if n is not much larger

than p , then there can be a lot of variability in the least squares fit, resulting in overfitting and consequently poor predictions on future observations not used in model training. And if $p > n$, then there is no longer a unique least squares coefficient estimate: the variance is *infinite* so the method cannot be used at all. By *constraining* or *shrinking* the estimated coefficients, we can often substantially reduce the variance at the cost of a negligible increase in bias. This can lead to substantial improvements in the accuracy with which we can predict the response for observations not used in model training.

- *Model Interpretability*: It is often the case that some or many of the variables used in a multiple regression model are in fact not associated with the response. Including such *irrelevant* variables leads to unnecessary complexity in the resulting model. By removing these variables—that is, by setting the corresponding coefficient estimates to zero—we can obtain a model that is more easily interpreted. Now least squares is extremely unlikely to yield any coefficient estimates that are exactly zero. In this chapter, we see some approaches for automatically performing *feature selection* or *variable selection*—that is, for excluding irrelevant variables from a multiple regression model.

feature
selection
variable
selection

There are many alternatives, both classical and modern, to using least squares to fit (6.1). In this chapter, we discuss three important classes of methods.

- *Subset Selection*. This approach involves identifying a subset of the p predictors that we believe to be related to the response. We then fit a model using least squares on the reduced set of variables.
- *Shrinkage*. This approach involves fitting a model involving all p predictors. However, the estimated coefficients are shrunk towards zero relative to the least squares estimates. This shrinkage (also known as *regularization*) has the effect of reducing variance. Depending on what type of shrinkage is performed, some of the coefficients may be estimated to be exactly zero. Hence, shrinkage methods can also perform variable selection.
- *Dimension Reduction*. This approach involves *projecting* the p predictors into an M -dimensional subspace, where $M < p$. This is achieved by computing M different *linear combinations*, or *projections*, of the variables. Then these M projections are used as predictors to fit a linear regression model by least squares.

In the following sections we describe each of these approaches in greater detail, along with their advantages and disadvantages. Although this chapter describes extensions and modifications to the linear model for regression seen in Chapter 3, the same concepts apply to other methods, such as the classification models seen in Chapter 4.

6.1 Subset Selection

In this section we consider some methods for selecting subsets of predictors. These include best subset and stepwise model selection procedures.

6.1.1 Best Subset Selection

To perform *best subset selection*, we fit a separate least squares regression for each possible combination of the p predictors. That is, we fit all p models that contain exactly one predictor, all $\binom{p}{2} = p(p-1)/2$ models that contain exactly two predictors, and so forth. We then look at all of the resulting models, with the goal of identifying the one that is *best*.

best subset
selection

The problem of selecting the *best model* from among the 2^p possibilities considered by best subset selection is not trivial. This is usually broken up into two stages, as described in Algorithm 6.1.

Algorithm 6.1 *Best subset selection*

1. Let $\mathcal{M}_0$ denote the *null model*, which contains no predictors. This model simply predicts the sample mean for each observation.
 2. For $k = 1, 2, \dots, p$:
 - (a) Fit all $\binom{p}{k}$ models that contain exactly k predictors.
 - (b) Pick the best among these $\binom{p}{k}$ models, and call it $\mathcal{M}_k$. Here *best* is defined as having the smallest RSS, or equivalently largest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

In Algorithm 6.1, Step 2 identifies the best model (on the training data) for each subset size, in order to reduce the problem from one of 2^p possible models to one of $p+1$ possible models. In Figure 6.1, these models form the lower frontier depicted in red.

Now in order to select a single best model, we must simply choose among these $p+1$ options. This task must be performed with care, because the RSS of these $p+1$ models decreases monotonically, and the R^2 increases monotonically, as the number of features included in the models increases. Therefore, if we use these statistics to select the best model, then we will always end up with a model involving all of the variables. The problem is that a low RSS or a high R^2 indicates a model with a low *training* error, whereas we wish to choose a model that has a low *test* error. (As shown in Chapter 2 in Figures 2.9–2.11, training error tends to be quite a bit smaller than test error, and a low training error by no means guarantees a low test error.) Therefore, in Step 3, we use cross-validated prediction

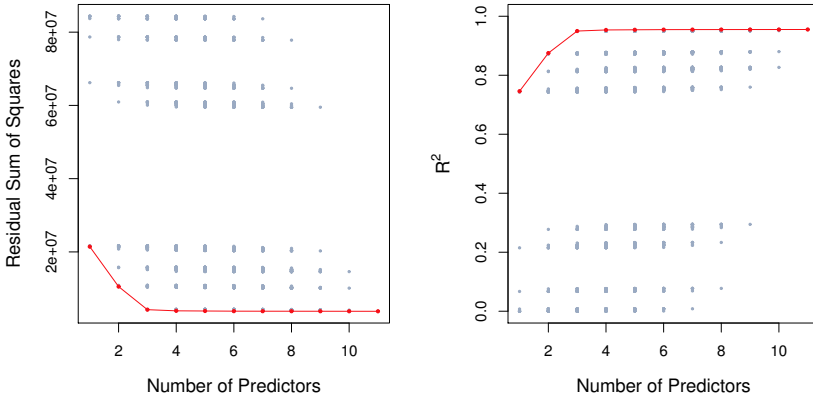


FIGURE 6.1. For each possible model containing a subset of the ten predictors in the **Credit** data set, the RSS and R^2 are displayed. The red frontier tracks the best model for a given number of predictors, according to RSS and R^2 . Though the data set contains only ten predictors, the x-axis ranges from 1 to 11, since one of the variables is categorical and takes on three values, leading to the creation of two dummy variables.

error, C_p , BIC, or adjusted R^2 in order to select among $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$. These approaches are discussed in Section 6.1.3.

An application of best subset selection is shown in Figure 6.1. Each plotted point corresponds to a least squares regression model fit using a different subset of the 10 predictors in the **Credit** data set, discussed in Chapter 3. Here the variable **region** is a three-level qualitative variable, and so is represented by two dummy variables, which are selected separately in this case. Hence, there are a total of 11 possible variables which can be included in the model. We have plotted the RSS and R^2 statistics for each model, as a function of the number of variables. The red curves connect the best models for each model size, according to RSS or R^2 . The figure shows that, as expected, these quantities improve as the number of variables increases; however, from the three-variable model on, there is little improvement in RSS and R^2 as a result of including additional predictors.

Although we have presented best subset selection here for least squares regression, the same ideas apply to other types of models, such as logistic regression. In the case of logistic regression, instead of ordering models by RSS in Step 2 of Algorithm 6.1, we instead use the *deviance*, a measure that plays the role of RSS for a broader class of models. The deviance is negative two times the maximized log-likelihood; the smaller the deviance, the better the fit.

deviance

While best subset selection is a simple and conceptually appealing approach, it suffers from computational limitations. The number of possible models that must be considered grows rapidly as p increases. In general,

there are 2^p models that involve subsets of p predictors. So if $p = 10$, then there are approximately 1,000 possible models to be considered, and if $p = 20$, then there are over one million possibilities! Consequently, best subset selection becomes computationally infeasible for values of p greater than around 40, even with extremely fast modern computers. There are computational shortcuts—so called branch-and-bound techniques—for eliminating some choices, but these have their limitations as p gets large. They also only work for least squares linear regression. We present computationally efficient alternatives to best subset selection next.

6.1.2 Stepwise Selection

For computational reasons, best subset selection cannot be applied with very large p . Best subset selection may also suffer from statistical problems when p is large. The larger the search space, the higher the chance of finding models that look good on the training data, even though they might not have any predictive power on future data. Thus an enormous search space can lead to overfitting and high variance of the coefficient estimates.

For both of these reasons, *stepwise* methods, which explore a far more restricted set of models, are attractive alternatives to best subset selection.

Forward Stepwise Selection

Forward stepwise selection is a computationally efficient alternative to best subset selection. While the best subset selection procedure considers all 2^p possible models containing subsets of the p predictors, forward stepwise considers a much smaller set of models. Forward stepwise selection begins with a model containing no predictors, and then adds predictors to the model, one-at-a-time, until all of the predictors are in the model. In particular, at each step the variable that gives the greatest *additional* improvement to the fit is added to the model. More formally, the forward stepwise selection procedure is given in Algorithm 6.2.

forward
stepwise
selection

Unlike best subset selection, which involved fitting 2^p models, forward stepwise selection involves fitting one null model, along with $p - k$ models in the k th iteration, for $k = 0, \dots, p - 1$. This amounts to a total of $1 + \sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ models. This is a substantial difference: when $p = 20$, best subset selection requires fitting 1,048,576 models, whereas forward stepwise selection requires fitting only 211 models.¹

¹Though forward stepwise selection considers $p(p + 1)/2 + 1$ models, it performs a *guided* search over model space, and so the *effective* model space considered contains substantially more than $p(p + 1)/2 + 1$ models.

Algorithm 6.2 *Forward stepwise selection*

1. Let $\mathcal{M}_0$ denote the *null* model, which contains no predictors.
 2. For $k = 0, \dots, p - 1$:
 - (a) Consider all $p - k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
 - (b) Choose the *best* among these $p - k$ models, and call it $\mathcal{M}_{k+1}$. Here *best* is defined as having smallest RSS or highest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

In Step 2(b) of Algorithm 6.2, we must identify the *best* model from among those $p - k$ that augment $\mathcal{M}_k$ with one additional predictor. We can do this by simply choosing the model with the lowest RSS or the highest R^2 . However, in Step 3, we must identify the best model among a set of models with different numbers of variables. This is more challenging, and is discussed in Section 6.1.3.

Forward stepwise selection's computational advantage over best subset selection is clear. Though forward stepwise tends to do well in practice, it is not guaranteed to find the best possible model out of all 2^p models containing subsets of the p predictors. For instance, suppose that in a given data set with $p = 3$ predictors, the best possible one-variable model contains X_1 , and the best possible two-variable model instead contains X_2 and X_3 . Then forward stepwise selection will fail to select the best possible two-variable model, because $\mathcal{M}_1$ will contain X_1 , so $\mathcal{M}_2$ must also contain X_1 together with one additional variable.

Table 6.1, which shows the first four selected models for best subset and forward stepwise selection on the **Credit** data set, illustrates this phenomenon. Both best subset selection and forward stepwise selection choose **rating** for the best one-variable model and then include **income** and **student** for the two- and three-variable models. However, best subset selection replaces **rating** by **cards** in the four-variable model, while forward stepwise selection must maintain **rating** in its four-variable model. In this example, Figure 6.1 indicates that there is not much difference between the three- and four-variable models in terms of RSS, so either of the four-variable models will likely be adequate.

Forward stepwise selection can be applied even in the high-dimensional setting where $n < p$; however, in this case, it is possible to construct submodels $\mathcal{M}_0, \dots, \mathcal{M}_{n-1}$ only, since each submodel is fit using least squares, which will not yield a unique solution if $p \geq n$.

# Variables	Best subset	Forward stepwise
One	rating	rating
Two	rating, income	rating, income
Three	rating, income, student	rating, income, student
Four	cards, income student, limit	rating, income, student, limit

TABLE 6.1. The first four selected models for best subset selection and forward stepwise selection on the **Credit** data set. The first three models are identical but the fourth models differ.

Backward Stepwise Selection

Like forward stepwise selection, *backward stepwise selection* provides an efficient alternative to best subset selection. However, unlike forward stepwise selection, it begins with the full least squares model containing all p predictors, and then iteratively removes the least useful predictor, one-at-a-time. Details are given in Algorithm 6.3.

backward
stepwise
selection

Algorithm 6.3 Backward stepwise selection

1. Let $\mathcal{M}_p$ denote the *full* model, which contains all p predictors.
 2. For $k = p, p - 1, \dots, 1$:
 - (a) Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$, for a total of $k - 1$ predictors.
 - (b) Choose the *best* among these k models, and call it $\mathcal{M}_{k-1}$. Here *best* is defined as having smallest RSS or highest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

Like forward stepwise selection, the backward selection approach searches through only $1 + p(p+1)/2$ models, and so can be applied in settings where p is too large to apply best subset selection.² Also like forward stepwise selection, backward stepwise selection is not guaranteed to yield the *best* model containing a subset of the p predictors.

Backward selection requires that the number of samples n is larger than the number of variables p (so that the full model can be fit). In contrast, forward stepwise can be used even when $n < p$, and so is the only viable subset method when p is very large.

²Like forward stepwise selection, backward stepwise selection performs a *guided* search over model space, and so effectively considers substantially more than $1 + p(p+1)/2$ models.

Hybrid Approaches

The best subset, forward stepwise, and backward stepwise selection approaches generally give similar but not identical models. As another alternative, hybrid versions of forward and backward stepwise selection are available, in which variables are added to the model sequentially, in analogy to forward selection. However, after adding each new variable, the method may also remove any variables that no longer provide an improvement in the model fit. Such an approach attempts to more closely mimic best subset selection while retaining the computational advantages of forward and backward stepwise selection.

6.1.3 Choosing the Optimal Model

Best subset selection, forward selection, and backward selection result in the creation of a set of models, each of which contains a subset of the p predictors. To apply these methods, we need a way to determine which of these models is *best*. As we discussed in Section 6.1.1, the model containing all of the predictors will always have the smallest RSS and the largest R^2 , since these quantities are related to the training error. Instead, we wish to choose a model with a low test error. As is evident here, and as we show in Chapter 2, the training error can be a poor estimate of the test error. Therefore, RSS and R^2 are not suitable for selecting the best model among a collection of models with different numbers of predictors.

In order to select the best model with respect to test error, we need to estimate this test error. There are two common approaches:

1. We can indirectly estimate test error by making an *adjustment* to the training error to account for the bias due to overfitting.
2. We can *directly* estimate the test error, using either a validation set approach or a cross-validation approach, as discussed in Chapter 5.

We consider both of these approaches below.

C_p , AIC, BIC, and Adjusted R^2

We show in Chapter 2 that the training set MSE is generally an underestimate of the test MSE. (Recall that $\text{MSE} = \text{RSS}/n$.) This is because when we fit a model to the training data using least squares, we specifically estimate the regression coefficients such that the training RSS (but not the test RSS) is as small as possible. In particular, the training error will decrease as more variables are included in the model, but the test error may not. Therefore, training set RSS and training set R^2 cannot be used to select from among a set of models with different numbers of variables.

However, a number of techniques for *adjusting* the training error for the model size are available. These approaches can be used to select among a set

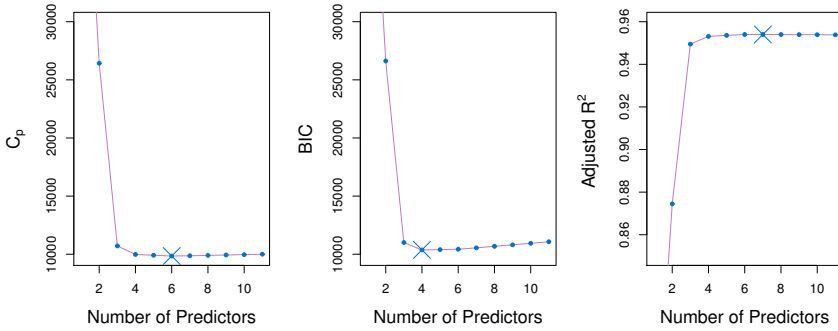


FIGURE 6.2. C_p , BIC, and adjusted R^2 are shown for the best models of each size for the **Credit** data set (the lower frontier in Figure 6.1). C_p and BIC are estimates of test MSE. In the middle plot we see that the BIC estimate of test error shows an increase after four variables are selected. The other two plots are rather flat after four variables are included.

of models with different numbers of variables. We now consider four such approaches: C_p , Akaike information criterion (AIC), Bayesian information criterion (BIC), and adjusted R^2 . Figure 6.2 displays C_p , BIC, and adjusted R^2 for the best model of each size produced by best subset selection on the **Credit** data set.

For a fitted least squares model containing d predictors, the C_p estimate of test MSE is computed using the equation

$$C_p = \frac{1}{n} (\text{RSS} + 2d\hat{\sigma}^2), \quad (6.2)$$

where $\hat{\sigma}^2$ is an estimate of the variance of the error ϵ associated with each response measurement in (6.1).³ Typically $\hat{\sigma}^2$ is estimated using the full model containing all predictors. Essentially, the C_p statistic adds a penalty of $2d\hat{\sigma}^2$ to the training RSS in order to adjust for the fact that the training error tends to underestimate the test error. Clearly, the penalty increases as the number of predictors in the model increases; this is intended to adjust for the corresponding decrease in training RSS. Though it is beyond the scope of this book, one can show that if $\hat{\sigma}^2$ is an unbiased estimate of σ^2 in (6.2), then C_p is an unbiased estimate of test MSE. As a consequence, the C_p statistic tends to take on a small value for models with a low test error, so when determining which of a set of models is best, we choose the model with the lowest C_p value. In Figure 6.2, C_p selects the six-variable model containing the predictors **income**, **limit**, **rating**, **cards**, **age** and **student**.

³Mallow's C_p is sometimes defined as $C'_p = \text{RSS}/\hat{\sigma}^2 + 2d - n$. This is equivalent to the definition given above in the sense that $C_p = \frac{1}{n}\hat{\sigma}^2(C'_p + n)$, and so the model with smallest C_p also has smallest C'_p .

C_p
Akaike
information
criterion
Bayesian
information
criterion
adjusted R^2

The AIC criterion is defined for a large class of models fit by maximum likelihood. In the case of the model (6.1) with Gaussian errors, maximum likelihood and least squares are the same thing. In this case AIC is given by

$$\text{AIC} = \frac{1}{n} (\text{RSS} + 2d\hat{\sigma}^2),$$

where, for simplicity, we have omitted irrelevant constants.⁴ Hence for least squares models, C_p and AIC are proportional to each other, and so only C_p is displayed in Figure 6.2.

BIC is derived from a Bayesian point of view, but ends up looking similar to C_p (and AIC) as well. For the least squares model with d predictors, the BIC is, up to irrelevant constants, given by

$$\text{BIC} = \frac{1}{n} (\text{RSS} + \log(n)d\hat{\sigma}^2). \quad (6.3)$$

Like C_p , the BIC will tend to take on a small value for a model with a low test error, and so generally we select the model that has the lowest BIC value. Notice that BIC replaces the $2d\hat{\sigma}^2$ used by C_p with a $\log(n)d\hat{\sigma}^2$ term, where n is the number of observations. Since $\log n > 2$ for any $n > 7$, the BIC statistic generally places a heavier penalty on models with many variables, and hence results in the selection of smaller models than C_p . In Figure 6.2, we see that this is indeed the case for the **Credit** data set; BIC chooses a model that contains only the four predictors **income**, **limit**, **cards**, and **student**. In this case the curves are very flat and so there does not appear to be much difference in accuracy between the four-variable and six-variable models.

The adjusted R^2 statistic is another popular approach for selecting among a set of models that contain different numbers of variables. Recall from Chapter 3 that the usual R^2 is defined as $1 - \text{RSS}/\text{TSS}$, where $\text{TSS} = \sum (y_i - \bar{y})^2$ is the *total sum of squares* for the response. Since RSS always decreases as more variables are added to the model, the R^2 always increases as more variables are added. For a least squares model with d variables, the adjusted R^2 statistic is calculated as

$$\text{Adjusted } R^2 = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}. \quad (6.4)$$

Unlike C_p , AIC, and BIC, for which a *small* value indicates a model with a low test error, a *large* value of adjusted R^2 indicates a model with a

⁴There are two formulas for AIC for least squares regression. The formula that we provide here requires an expression for σ^2 , which we obtain using the full model containing all predictors. The second formula is appropriate when σ^2 is unknown and we do not want to explicitly estimate it; that formula has a $\log(\text{RSS})$ term instead of an RSS term. Detailed derivations of these two formulas are outside of the scope of this book.

small test error. Maximizing the adjusted R^2 is equivalent to minimizing $\frac{\text{RSS}}{n-d-1}$. While RSS always decreases as the number of variables in the model increases, $\frac{\text{RSS}}{n-d-1}$ may increase or decrease, due to the presence of d in the denominator.

The intuition behind the adjusted R^2 is that once all of the correct variables have been included in the model, adding additional *noise* variables will lead to only a very small decrease in RSS. Since adding noise variables leads to an increase in d , such variables will lead to an increase in $\frac{\text{RSS}}{n-d-1}$, and consequently a decrease in the adjusted R^2 . Therefore, in theory, the model with the largest adjusted R^2 will have only correct variables and no noise variables. Unlike the R^2 statistic, the adjusted R^2 statistic *pays a price* for the inclusion of unnecessary variables in the model. Figure 6.2 displays the adjusted R^2 for the **Credit** data set. Using this statistic results in the selection of a model that contains seven variables, adding **own** to the model selected by C_p and AIC.

C_p , AIC, and BIC all have rigorous theoretical justifications that are beyond the scope of this book. These justifications rely on asymptotic arguments (scenarios where the sample size n is very large). Despite its popularity, and even though it is quite intuitive, the adjusted R^2 is not as well motivated in statistical theory as AIC, BIC, and C_p . All of these measures are simple to use and compute. Here we have presented their formulas in the case of a linear model fit using least squares; however, AIC and BIC can also be defined for more general types of models.

Validation and Cross-Validation

As an alternative to the approaches just discussed, we can directly estimate the test error using the validation set and cross-validation methods discussed in Chapter 5. We can compute the validation set error or the cross-validation error for each model under consideration, and then select the model for which the resulting estimated test error is smallest. This procedure has an advantage relative to AIC, BIC, C_p , and adjusted R^2 , in that it provides a direct estimate of the test error, and makes fewer assumptions about the true underlying model. It can also be used in a wider range of model selection tasks, even in cases where it is hard to pinpoint the model degrees of freedom (e.g. the number of predictors in the model) or hard to estimate the error variance σ^2 .

In the past, performing cross-validation was computationally prohibitive for many problems with large p and/or large n , and so AIC, BIC, C_p , and adjusted R^2 were more attractive approaches for choosing among a set of models. However, nowadays with fast computers, the computations required to perform cross-validation are hardly ever an issue. Thus, cross-validation is a very attractive approach for selecting from among a number of models under consideration.

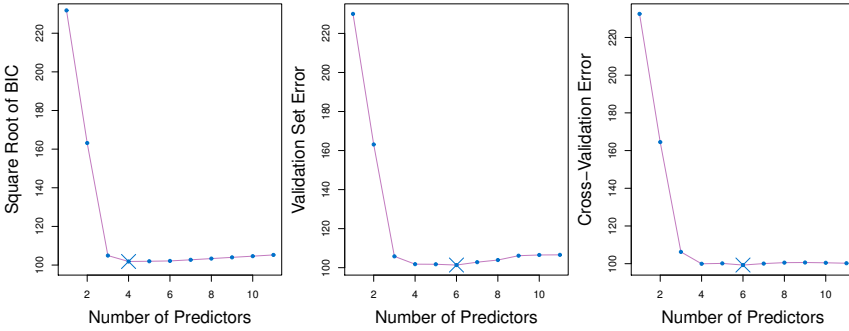


FIGURE 6.3. For the **Credit** data set, three quantities are displayed for the best model containing d predictors, for d ranging from 1 to 11. The overall best model, based on each of these quantities, is shown as a blue cross. Left: Square root of BIC. Center: Validation set errors. Right: Cross-validation errors.

Figure 6.3 displays, as a function of d , the BIC, validation set errors, and cross-validation errors on the **Credit** data, for the best d -variable model. The validation errors were calculated by randomly selecting three-quarters of the observations as the training set, and the remainder as the validation set. The cross-validation errors were computed using $k = 10$ folds. In this case, the validation and cross-validation methods both result in a six-variable model. However, all three approaches suggest that the four-, five-, and six-variable models are roughly equivalent in terms of their test errors.

In fact, the estimated test error curves displayed in the center and right-hand panels of Figure 6.3 are quite flat. While a three-variable model clearly has lower estimated test error than a two-variable model, the estimated test errors of the 3- to 11-variable models are quite similar. Furthermore, if we repeated the validation set approach using a different split of the data into a training set and a validation set, or if we repeated cross-validation using a different set of cross-validation folds, then the precise model with the lowest estimated test error would surely change. In this setting, we can select a model using the *one-standard-error rule*. We first calculate the standard error of the estimated test MSE for each model size, and then select the smallest model for which the estimated test error is within one standard error of the lowest point on the curve. The rationale here is that if a set of models appear to be more or less equally good, then we might as well choose the simplest model—that is, the model with the smallest number of predictors. In this case, applying the one-standard-error rule to the validation set or cross-validation approach leads to selection of the three-variable model.

one-
standard-
error
rule

6.2 Shrinkage Methods

The subset selection methods described in Section 6.1 involve using least squares to fit a linear model that contains a subset of the predictors. As an alternative, we can fit a model containing all p predictors using a technique that *constrains* or *regularizes* the coefficient estimates, or equivalently, that *shrinks* the coefficient estimates towards zero. It may not be immediately obvious why such a constraint should improve the fit, but it turns out that shrinking the coefficient estimates can significantly reduce their variance. The two best-known techniques for shrinking the regression coefficients towards zero are *ridge regression* and the *lasso*.

6.2.1 Ridge Regression

Recall from Chapter 3 that the least squares fitting procedure estimates $\beta_0, \beta_1, \dots, \beta_p$ using the values that minimize

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2.$$

Ridge regression is very similar to least squares, except that the coefficients are estimated by minimizing a slightly different quantity. In particular, the ridge regression coefficient estimates $\hat{\beta}^R$ are the values that minimize

ridge
regression

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.5)$$

where $\lambda \geq 0$ is a *tuning parameter*, to be determined separately. Equation 6.5 trades off two different criteria. As with least squares, ridge regression seeks coefficient estimates that fit the data well, by making the RSS small. However, the second term, $\lambda \sum_j \beta_j^2$, called a *shrinkage penalty*, is small when $\beta_1, \dots, \beta_p$ are close to zero, and so it has the effect of *shrinking* the estimates of β_j towards zero. The tuning parameter λ serves to control the relative impact of these two terms on the regression coefficient estimates. When $\lambda = 0$, the penalty term has no effect, and ridge regression will produce the least squares estimates. However, as $\lambda \rightarrow \infty$, the impact of the shrinkage penalty grows, and the ridge regression coefficient estimates will approach zero. Unlike least squares, which generates only one set of coefficient estimates, ridge regression will produce a different set of coefficient estimates, $\hat{\beta}_\lambda^R$, for each value of λ . Selecting a good value for λ is critical; we defer this discussion to Section 6.2.3, where we use cross-validation.

tuning
parameter

shrinkage
penalty

Note that in (6.5), the shrinkage penalty is applied to $\beta_1, \dots, \beta_p$, but not to the intercept β_0 . We want to shrink the estimated association of

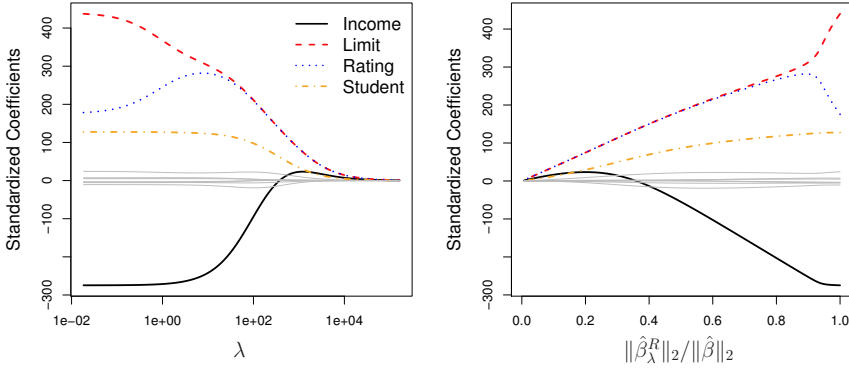


FIGURE 6.4. The standardized ridge regression coefficients are displayed for the **Credit** data set, as a function of λ and $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$.

each variable with the response; however, we do not want to shrink the intercept, which is simply a measure of the mean value of the response when $x_{i1} = x_{i2} = \dots = x_{ip} = 0$. If we assume that the variables—that is, the columns of the data matrix $\mathbf{X}$ —have been centered to have mean zero before ridge regression is performed, then the estimated intercept will take the form $\hat{\beta}_0 = \bar{y} = \sum_{i=1}^n y_i / n$.

An Application to the Credit Data

In Figure 6.4, the ridge regression coefficient estimates for the **Credit** data set are displayed. In the left-hand panel, each curve corresponds to the ridge regression coefficient estimate for one of the ten variables, plotted as a function of λ . For example, the black solid line represents the ridge regression estimate for the **income** coefficient, as λ is varied. At the extreme left-hand side of the plot, λ is essentially zero, and so the corresponding ridge coefficient estimates are the same as the usual least squares estimates. But as λ increases, the ridge coefficient estimates shrink towards zero. When λ is extremely large, then all of the ridge coefficient estimates are basically zero; this corresponds to the *null model* that contains no predictors. In this plot, the **income**, **limit**, **rating**, and **student** variables are displayed in distinct colors, since these variables tend to have by far the largest coefficient estimates. While the ridge coefficient estimates tend to decrease in aggregate as λ increases, individual coefficients, such as **rating** and **income**, may occasionally increase as λ increases.

The right-hand panel of Figure 6.4 displays the same ridge coefficient estimates as the left-hand panel, but instead of displaying λ on the x -axis, we now display $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$, where $\hat{\beta}$ denotes the vector of least squares coefficient estimates. The notation $\|\beta\|_2$ denotes the ℓ_2 norm (pronounced “ell 2”) of a vector, and is defined as $\|\beta\|_2 = \sqrt{\sum_{j=1}^p \beta_j^2}$. It measures

the distance of β from zero. As λ increases, the ℓ_2 norm of $\hat{\beta}_\lambda^R$ will *always* decrease, and so will $\|\hat{\beta}_\lambda^R\|_2/\|\hat{\beta}\|_2$. The latter quantity ranges from 1 (when $\lambda = 0$, in which case the ridge regression coefficient estimate is the same as the least squares estimate, and so their ℓ_2 norms are the same) to 0 (when $\lambda = \infty$, in which case the ridge regression coefficient estimate is a vector of zeros, with ℓ_2 norm equal to zero). Therefore, we can think of the x -axis in the right-hand panel of Figure 6.4 as the amount that the ridge regression coefficient estimates have been shrunk towards zero; a small value indicates that they have been shrunk very close to zero.

The standard least squares coefficient estimates discussed in Chapter 3 are *scale equivariant*: multiplying X_j by a constant c simply leads to a scaling of the least squares coefficient estimates by a factor of $1/c$. In other words, regardless of how the j th predictor is scaled, $X_j\hat{\beta}_j$ will remain the same. In contrast, the ridge regression coefficient estimates can change *substantially* when multiplying a given predictor by a constant. For instance, consider the **income** variable, which is measured in dollars. One could reasonably have measured income in thousands of dollars, which would result in a reduction in the observed values of **income** by a factor of 1,000. Now due to the sum of squared coefficients term in the ridge regression formulation (6.5), such a change in scale will not simply cause the ridge regression coefficient estimate for **income** to change by a factor of 1,000. In other words, $X_j\hat{\beta}_{j,\lambda}^R$ will depend not only on the value of λ , but also on the scaling of the j th predictor. In fact, the value of $X_j\hat{\beta}_{j,\lambda}^R$ may even depend on the scaling of the *other* predictors! Therefore, it is best to apply ridge regression after *standardizing the predictors*, using the formula

scale
equivariant

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}}, \quad (6.6)$$

so that they are all on the same scale. In (6.6), the denominator is the estimated standard deviation of the j th predictor. Consequently, all of the standardized predictors will have a standard deviation of one. As a result the final fit will not depend on the scale on which the predictors are measured. In Figure 6.4, the y -axis displays the standardized ridge regression coefficient estimates—that is, the coefficient estimates that result from performing ridge regression using standardized predictors.

Why Does Ridge Regression Improve Over Least Squares?

Ridge regression's advantage over least squares is rooted in the *bias-variance trade-off*. As λ increases, the flexibility of the ridge regression fit decreases, leading to decreased variance but increased bias. This is illustrated in the left-hand panel of Figure 6.5, using a simulated data set containing $p = 45$ predictors and $n = 50$ observations. The green curve in the left-hand panel of Figure 6.5 displays the variance of the ridge regression predictions as a

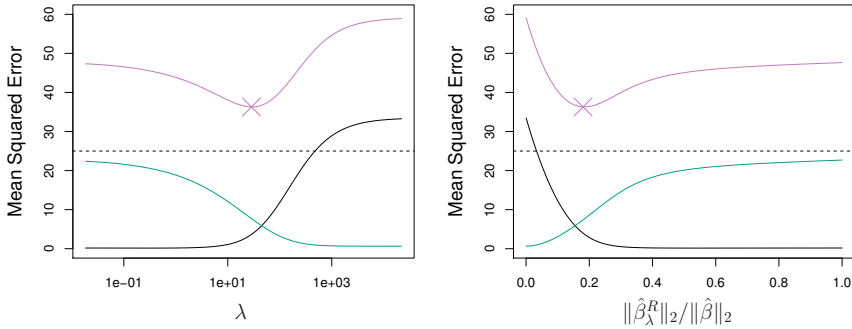


FIGURE 6.5. Squared bias (black), variance (green), and test mean squared error (purple) for the ridge regression predictions on a simulated data set, as a function of λ and $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$. The horizontal dashed lines indicate the minimum possible MSE. The purple crosses indicate the ridge regression models for which the MSE is smallest.

function of λ . At the least squares coefficient estimates, which correspond to ridge regression with $\lambda = 0$, the variance is high but there is no bias. But as λ increases, the shrinkage of the ridge coefficient estimates leads to a substantial reduction in the variance of the predictions, at the expense of a slight increase in bias. Recall that the test mean squared error (MSE), plotted in purple, is closely related to the variance plus the squared bias. For values of λ up to about 10, the variance decreases rapidly, with very little increase in bias, plotted in black. Consequently, the MSE drops considerably as λ increases from 0 to 10. Beyond this point, the decrease in variance due to increasing λ slows, and the shrinkage on the coefficients causes them to be significantly underestimated, resulting in a large increase in the bias. The minimum MSE is achieved at approximately $\lambda = 30$. Interestingly, because of its high variance, the MSE associated with the least squares fit, when $\lambda = 0$, is almost as high as that of the null model for which all coefficient estimates are zero, when $\lambda = \infty$. However, for an intermediate value of λ , the MSE is considerably lower.

The right-hand panel of Figure 6.5 displays the same curves as the left-hand panel, this time plotted against the ℓ_2 norm of the ridge regression coefficient estimates divided by the ℓ_2 norm of the least squares estimates. Now as we move from left to right, the fits become more flexible, and so the bias decreases and the variance increases.

In general, in situations where the relationship between the response and the predictors is close to linear, the least squares estimates will have low bias but may have high variance. This means that a small change in the training data can cause a large change in the least squares coefficient estimates. In particular, when the number of variables p is almost as large as the number of observations n , as in the example in Figure 6.5, the least squares estimates will be extremely variable. And if $p > n$, then the

least squares estimates do not even have a unique solution, whereas ridge regression can still perform well by trading off a small increase in bias for a large decrease in variance. Hence, ridge regression works best in situations where the least squares estimates have high variance.

Ridge regression also has substantial computational advantages over best subset selection, which requires searching through 2^p models. As we discussed previously, even for moderate values of p , such a search can be computationally infeasible. In contrast, for any fixed value of λ , ridge regression only fits a single model, and the model-fitting procedure can be performed quite quickly. In fact, one can show that the computations required to solve (6.5), *simultaneously for all values of λ* , are almost identical to those for fitting a model using least squares.

6.2.2 The Lasso

Ridge regression does have one obvious disadvantage. Unlike best subset, forward stepwise, and backward stepwise selection, which will generally select models that involve just a subset of the variables, ridge regression will include all p predictors in the final model. The penalty $\lambda \sum \beta_j^2$ in (6.5) will shrink all of the coefficients towards zero, but it will not set any of them exactly to zero (unless $\lambda = \infty$). This may not be a problem for prediction accuracy, but it can create a challenge in model interpretation in settings in which the number of variables p is quite large. For example, in the **Credit** data set, it appears that the most important variables are **income**, **limit**, **rating**, and **student**. So we might wish to build a model including just these predictors. However, ridge regression will always generate a model involving all ten predictors. Increasing the value of λ will tend to reduce the magnitudes of the coefficients, but will not result in exclusion of any of the variables.

The *lasso* is a relatively recent alternative to ridge regression that overcomes this disadvantage. The lasso coefficients, $\hat{\beta}_\lambda^L$, minimize the quantity lasso

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|. \quad (6.7)$$

Comparing (6.7) to (6.5), we see that the lasso and ridge regression have similar formulations. The only difference is that the β_j^2 term in the ridge regression penalty (6.5) has been replaced by $|\beta_j|$ in the lasso penalty (6.7). In statistical parlance, the lasso uses an ℓ_1 (pronounced “ell 1”) penalty instead of an ℓ_2 penalty. The ℓ_1 norm of a coefficient vector β is given by $\|\beta\|_1 = \sum |\beta_j|$.

As with ridge regression, the lasso shrinks the coefficient estimates towards zero. However, in the case of the lasso, the ℓ_1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when

the tuning parameter λ is sufficiently large. Hence, much like best subset selection, the lasso performs *variable selection*. As a result, models generated from the lasso are generally much easier to interpret than those produced by ridge regression. We say that the lasso yields *sparse* models—that is, models that involve only a subset of the variables. As in ridge regression, selecting a good value of λ for the lasso is critical; we defer this discussion to Section 6.2.3, where we use cross-validation.

As an example, consider the coefficient plots in Figure 6.6, which are generated from applying the lasso to the **Credit** data set. When $\lambda = 0$, then the lasso simply gives the least squares fit, and when λ becomes sufficiently large, the lasso gives the null model in which all coefficient estimates equal zero. However, in between these two extremes, the ridge regression and lasso models are quite different from each other. Moving from left to right in the right-hand panel of Figure 6.6, we observe that at first the lasso results in a model that contains only the **rating** predictor. Then **student** and **limit** enter the model almost simultaneously, shortly followed by **income**. Eventually, the remaining variables enter the model. Hence, depending on the value of λ , the lasso can produce a model involving any number of variables. In contrast, ridge regression will always include all of the variables in the model, although the magnitude of the coefficient estimates will depend on λ .

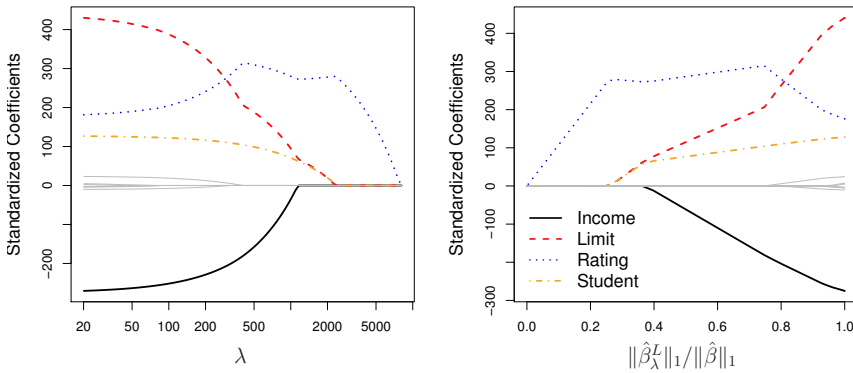


FIGURE 6.6. The standardized lasso coefficients on the **Credit** data set are shown as a function of λ and $\|\hat{\beta}_\lambda^L\|_1 / \|\hat{\beta}\|_1$.

Another Formulation for Ridge Regression and the Lasso

One can show that the lasso and ridge regression coefficient estimates solve the problems

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s \quad (6.8)$$

and

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 \leq s, \quad (6.9)$$

respectively. In other words, for every value of λ , there is some s such that the Equations (6.7) and (6.8) will give the same lasso coefficient estimates. Similarly, for every value of λ there is a corresponding s such that Equations (6.5) and (6.9) will give the same ridge regression coefficient estimates. When $p = 2$, then (6.8) indicates that the lasso coefficient estimates have the smallest RSS out of all points that lie within the diamond defined by $|\beta_1| + |\beta_2| \leq s$. Similarly, the ridge regression estimates have the smallest RSS out of all points that lie within the circle defined by $\beta_1^2 + \beta_2^2 \leq s$.

We can think of (6.8) as follows. When we perform the lasso we are trying to find the set of coefficient estimates that lead to the smallest RSS, subject to the constraint that there is a *budget* s for how large $\sum_{j=1}^p |\beta_j|$ can be. When s is extremely large, then this budget is not very restrictive, and so the coefficient estimates can be large. In fact, if s is large enough that the least squares solution falls within the budget, then (6.8) will simply yield the least squares solution. In contrast, if s is small, then $\sum_{j=1}^p |\beta_j|$ must be small in order to avoid violating the budget. Similarly, (6.9) indicates that when we perform ridge regression, we seek a set of coefficient estimates such that the RSS is as small as possible, subject to the requirement that $\sum_{j=1}^p \beta_j^2$ not exceed the budget s .

The formulations (6.8) and (6.9) reveal a close connection between the lasso, ridge regression, and best subset selection. Consider the problem

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p I(\beta_j \neq 0) \leq s. \quad (6.10)$$

Here $I(\beta_j \neq 0)$ is an indicator variable: it takes on a value of 1 if $\beta_j \neq 0$, and equals zero otherwise. Then (6.10) amounts to finding a set of coefficient estimates such that RSS is as small as possible, subject to the constraint that no more than s coefficients can be nonzero. The problem (6.10) is

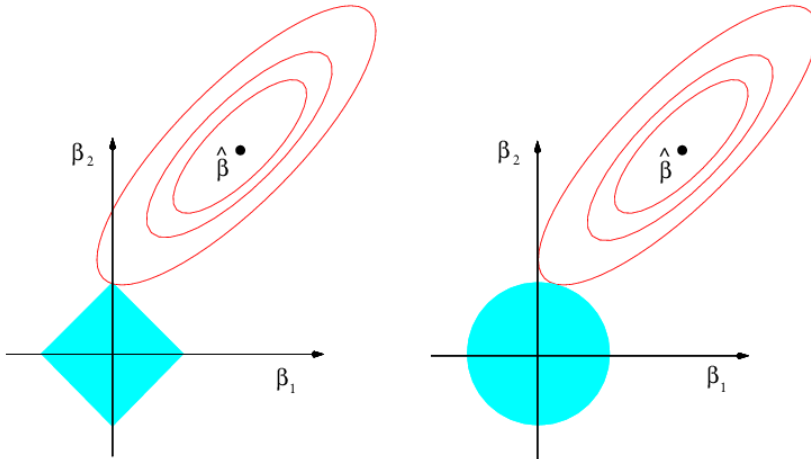


FIGURE 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

equivalent to best subset selection. Unfortunately, solving (6.10) is computationally infeasible when p is large, since it requires considering all $\binom{p}{s}$ models containing s predictors. Therefore, we can interpret ridge regression and the lasso as computationally feasible alternatives to best subset selection that replace the intractable form of the budget in (6.10) with forms that are much easier to solve. Of course, the lasso is much more closely related to best subset selection, since the lasso performs feature selection for s sufficiently small in (6.8), while ridge regression does not.

The Variable Selection Property of the Lasso

Why is it that the lasso, unlike ridge regression, results in coefficient estimates that are exactly equal to zero? The formulations (6.8) and (6.9) can be used to shed light on the issue. Figure 6.7 illustrates the situation. The least squares solution is marked as $\hat{\beta}$, while the blue diamond and circle represent the lasso and ridge regression constraints in (6.8) and (6.9), respectively. If s is sufficiently large, then the constraint regions will contain $\hat{\beta}$, and so the ridge regression and lasso estimates will be the same as the least squares estimates. (Such a large value of s corresponds to $\lambda = 0$ in (6.5) and (6.7).) However, in Figure 6.7 the least squares estimates lie outside of the diamond and the circle, and so the least squares estimates are not the same as the lasso and ridge regression estimates.

Each of the ellipses centered around $\hat{\beta}$ represents a *contour*: this means that all of the points on a particular ellipse have the same RSS value. As

contour

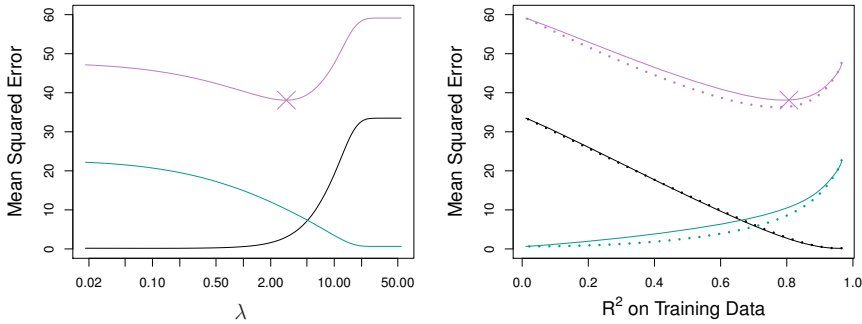


FIGURE 6.8. Left: Plots of squared bias (black), variance (green), and test MSE (purple) for the lasso on a simulated data set. Right: Comparison of squared bias, variance, and test MSE between lasso (solid) and ridge (dotted). Both are plotted against their R^2 on the training data, as a common form of indexing. The crosses in both plots indicate the lasso model for which the MSE is smallest.

the ellipses expand away from the least squares coefficient estimates, the RSS increases. Equations (6.8) and (6.9) indicate that the lasso and ridge regression coefficient estimates are given by the first point at which an ellipse contacts the constraint region. Since ridge regression has a circular constraint with no sharp points, this intersection will not generally occur on an axis, and so the ridge regression coefficient estimates will be exclusively non-zero. However, the lasso constraint has *corners* at each of the axes, and so the ellipse will often intersect the constraint region at an axis. When this occurs, one of the coefficients will equal zero. In higher dimensions, many of the coefficient estimates may equal zero simultaneously. In Figure 6.7, the intersection occurs at $\beta_1 = 0$, and so the resulting model will only include β_2 .

In Figure 6.7, we considered the simple case of $p = 2$. When $p = 3$, then the constraint region for ridge regression becomes a sphere, and the constraint region for the lasso becomes a polyhedron. When $p > 3$, the constraint for ridge regression becomes a hypersphere, and the constraint for the lasso becomes a polytope. However, the key ideas depicted in Figure 6.7 still hold. In particular, the lasso leads to feature selection when $p > 2$ due to the sharp corners of the polyhedron or polytope.

Comparing the Lasso and Ridge Regression

It is clear that the lasso has a major advantage over ridge regression, in that it produces simpler and more interpretable models that involve only a subset of the predictors. However, which method leads to better prediction accuracy? Figure 6.8 displays the variance, squared bias, and test MSE of the lasso applied to the same simulated data as in Figure 6.5. Clearly the lasso leads to qualitatively similar behavior to ridge regression, in that as λ

Lesson Notes

MODULE 3 | LESSON 3

TREES

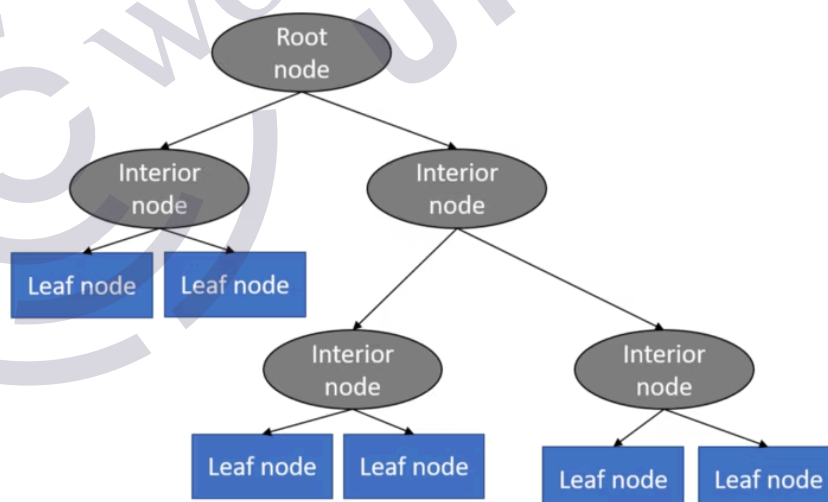
Reading Time	90 minutes
Prior Knowledge	Logistic regression
Keywords	Gini Impurity, Nodes, Homogeneous nodes, Boosting, ROC Curve

We have looked at one of the tools for classification, namely Naïve Bayes classification. This method builds on the framework of Bayesian statistics and makes use of a prior and likelihood distribution. These distributions are based on the data we train the model on, and we make the naïve assumption of independence between predictors. Oftentimes, this is not the case with real-world problems. Let us now look at an alternative method that does not make such an assumption and is widely used in practice.

1. Classification Trees

To understand tree-based models, we need to understand the basic building blocks, which are decision trees. The components of decision trees are shown in figure 1. These are nodes that have specific names based on their position in the flow diagram. By following the flow from top to bottom, we initially have the root, interior, and finally the leaf nodes. The splits into left/right directions are called **branches**.

Figure 1: The Basic Components and Structure of a Decision Tree.



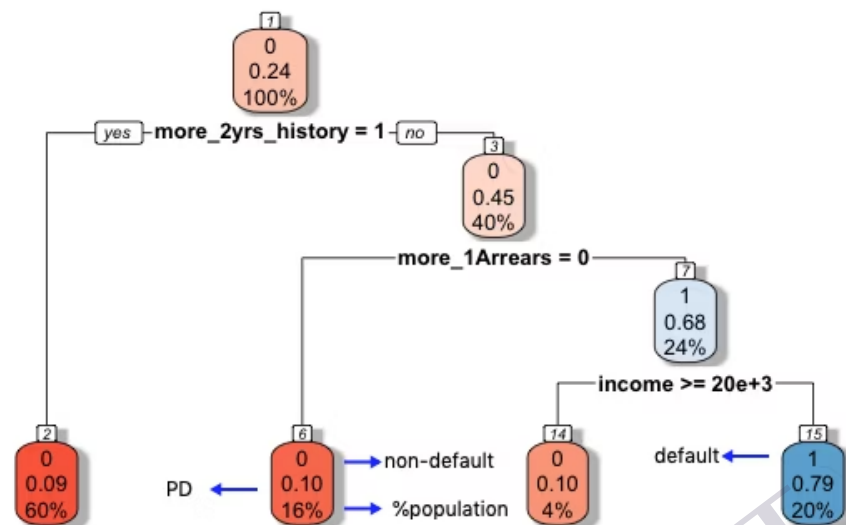
Source: python-course.edu, Decision trees in python, June 2022. <https://python-course.eu/machine-learning/decision-trees-in-python.php>.)

To conceptualize the application of the decision tree, we'll look at a credit risk default example for a bank loan based on just three explanatory variables. The target variable classes are 0: non-default, 1: default. Keep in mind that this example is to convey the idea of a decision tree as the banking industry uses much more explanatory variables/predictors in practice.

The first of the three predictors is a binary variable that flags an applicant if they have more than two years of credit history in the credit bureau. This would be valuable to a bank as it allows the bank to assess how the client performs financially. The second predictor is also a binary variable such that it is 1 if the client has had more than 1 missed

payment for other loans in the last 2 years and 0 otherwise. The third predictor is the client's available monthly income. Assume the currency is in South African rand. The figure identifies the rules that would lead us to the particular target classes. For instance, figure 2 shows that applicants with more than two years of credit history have a 9% probability of defaulting (PD) by looking at the path that leads to the bottom left-most leaf node.

Figure 2: A decision tree model applied to credit risk application data on 3 predictors.



This population makes up 60% of the population. Following the path to the right, we look at those with at most two years of credit history. This population has a PD of 45%. Splitting this population further into those that had more than 1 arrears in their two-year credit history and have an income under 20,000 rand, there is a 79% chance of default. This group makes up 20% of the population and has a high chance of default. It is important to assess the logic and the sensibility of the rules as oftentimes the analyst will need to explain this to stakeholders. In this example, the data confirms our suspicion that those with lower incomes, little credit history and history of missing more than one payment are likely to default.

Now that we understand how to use decision trees, let us look at how the tree is trained on data. To do this, we first examine some commonly used metrics to determine the splitting at branches.

2. Gini Impurity

Decision trees split the samples by sequentially selecting the predictors according to a calculated measure. The measure we look at is **Gini impurity**, named after Corrado Gini. It is given by $G = 1 - \sum_k (p_k)^2$, (3.1) where the sum is over all possible $k = 1, \dots, c$ target classes. In our example, this is seen in figure 2 for $k = 1, 2$. The terms p_k are the proportion of the class in that particular node. For instance, figure 2 splits the more than two years of credit history variable first. This feature has only two classes and we have the default percentage of the classes in table 1.

Table 1

More than 2 yrs credit history	% default
YES	9%
NO	91%

We therefore have from (3.1),

$$G = 1 - [(0.09^2) + (0.91^2)] \\ = 0.164$$

The idea behind Gini impurity is to obtain as pure a node as possible, thereby minimizing the risk of misclassification. We want as low a Gini impurity as possible to achieve this. You can see that the left-most leaf node has a very low percentage of default population. Similarly, the inner node for those with ≤ 2 yrs credit history has a Gini impurity of C

Table 2

Leaf	Gini Impurity
≤ 2yrs history, more than 1 arrears, at least R20k income	18%
≤ 2yrs history, more than 1 arrears, and < R20k income	33%

The variable at the root node is chosen to produce the purest node, i.e., the lowest Gini impurity. The decision tree has to therefore split on all predictors to determine this.

Another commonly used measure for splitting a tree is entropy. Entropy or Gini impurity is used to maximize **information gain**. The reader is referred to Hull (133) should they be interested in understanding this. We've seen trees in a classification problem, but keep in mind that they can also be used for a continuous target variable. These are regression problems, which we cover next.

For an application of decision trees with some mock data, this video should clarify the concepts discussed above:

12:55

[Access video transcript here](#)

3. Regression Trees

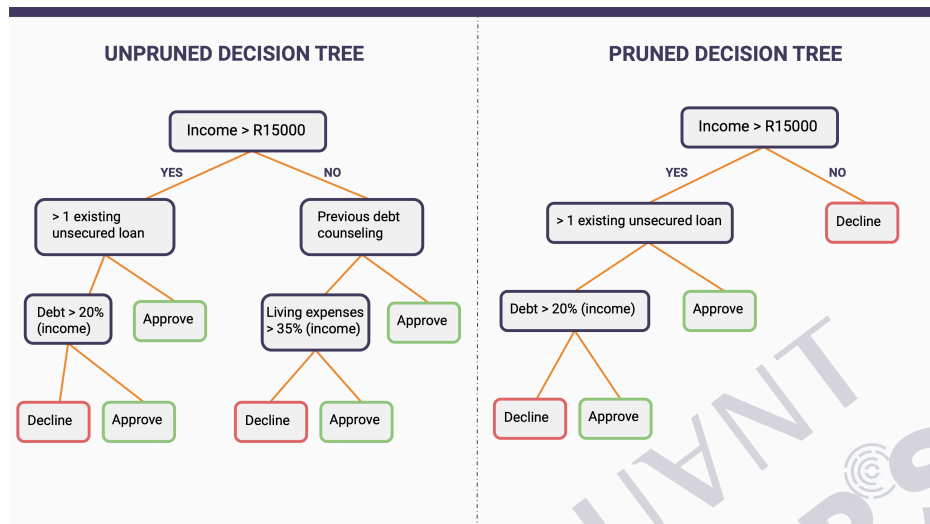
Thus far, we have referred to only classification problems; however, we may in practice be faced with a continuous target variable. In this instance, we would be interested in predicting the amount or value in a financial problem. In this type of problem, a regression tree would be applicable. The difference in the chart as compared to figure 2 is that the leaf nodes would show the mean value of the target variable. The data is split for each predictor at every point, and the split with the lowest variation between the predicted and actual target values is chosen as the splitting point. The measure of variation is the sum of squares, which should be familiar from linear regression. For any predictor i we have (3.2):

$$SSE = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

with y_i the predicted value and $\bar{y}_1$ and $\bar{y}_2$ the mean value of the left and right side of the split. The split that produces the lowest SSE would be chosen. Trees, like other machine learning algorithms, are susceptible to overfitting; however, there are ways to minimize this. One of them is called pruning.

Pruning aims to reduce the size of a decision tree, thus reducing overfitting. This is done by reducing sections of the tree that are non-critical. We ideally want the tree to generalize unseen samples well. One of the pruning strategies involves producing a tree with small samples in the leaf nodes, thereby reducing the maximum depth of the tree and eliminating those overfitted leaves. This is an example of post-pruning. With pre-pruning, we specify parameters to prevent leaves with too few samples; an example would be maximum depth or minimum Gini. This early stoppage can be efficient as there are fewer computations. Figure 3 illustrates an unpruned vs. pruned tree.

Figure 3: Unpruned vs. Pruned Tree



One of the most simplistic predictive classification algorithms is logistic regression, and it is worth mentioning the comparisons to decision trees.

4. Logistic Regression vs. Decision Trees

Both methods deal with classification problems and are easily interpreted. Decision trees can split the data non-linearly; hence, the data does not have to be linearly separable.

Figure 4: Non-Linearly Separable Data

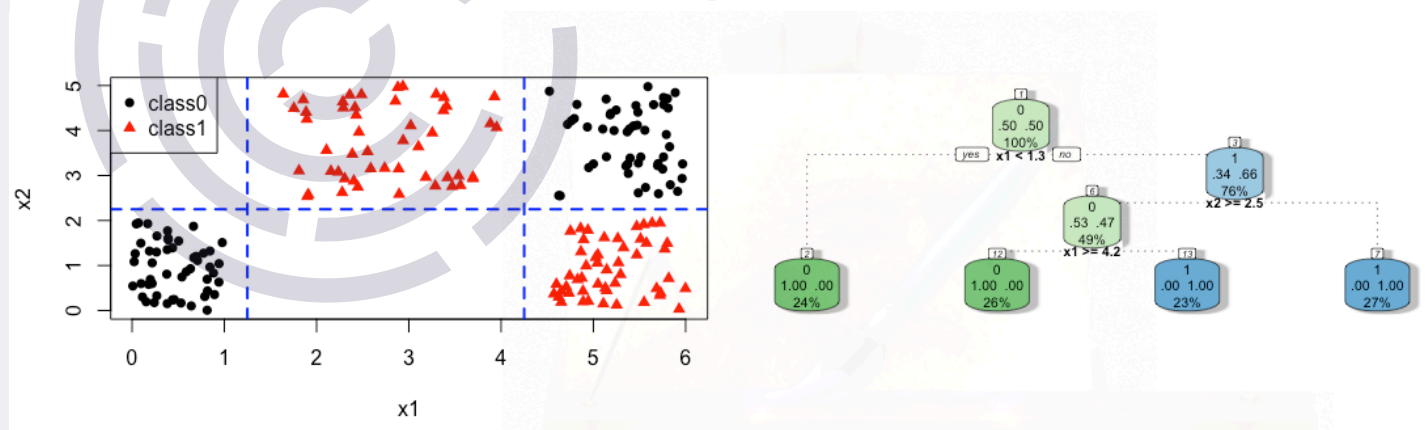


Figure 4 illustrates the non-linearly separable data of which a decision tree is more effective at class predicting. Logistic regression handles linearly separable data well. Decision trees split predictors sequentially; therefore, it is also not necessary to scale the data, unlike logistic regression. Logistic regression is also sensitive to outliers and skewness of data, which decision trees are not. With few variables in decision trees, the variable interaction is easily interpretable and shown. Quite often, a decision tree model is chosen over the logistic regression model; however, some financial institutions are regulated to use logistic regression models.

In the next section, we will look at a modified tree-based algorithm that leverages other trees to improve predictive performance.

5. Boosting

Boosting is a tree-based method that combines the power of other trees, specifically weak-learning trees, which we refer to as **weak learners**. These weak learners basically have a root node and a split on that predictor. The process combines these weak learners in a sequential manner, such that each weak learner improves upon the previous learner, meaning the final model is improved along the way. It is because of this sequential nature that the training takes longer than other parallel methods. Suppose at iteration i we have a set of predictions $P_i(X)$ from a weak learner based on a set of predictors X . The predictions at iterations can be related as:

$$P_{i+1}(X) = P_i(X) + T_i(X)$$

where $T_i(X)$ is a tree-based learner introduced at iteration i to improve on $P_i(X)$. Residuals are examined to determine adjustments to the tuning parameters of T . These residuals can be expressed as,

$$\epsilon = Y - (P_i(X) + T_i(X))$$

A popular gradient boosting algorithm used is Xgboost. These algorithms tend to easily overfit and are sensitive to noise in the data. Results are also very sensitive to parameter tuning. For a consolidation of the above concepts with mock data, watch the following video:



Thus far, we have looked at accuracy for assessing model performance. In the next section, we will examine a commonly used metric and diagram to assess model performance.

6. Receiver Operating Curve

We'll discuss a measure used for classification models, specifically the receiver operating curve (ROC) and area under the curve (AUC). These measures actually stem from the Gini coefficient and Lorentz curve, which were initially used to show income inequality in countries. We will not explore the Gini coefficient as we will stick to just the machine learning application of it. However, should the reader be interested in the Gini coefficient, we refer to Gini (1). We first show the confusion matrix for a binary classifier in table 3, which is basically a cross-tabulation of the predicted classes vs. the actual classes:

	Actual (0)	Actual (1)
Predicted (0)	TN	FN
Predicted (1)	FP	TP

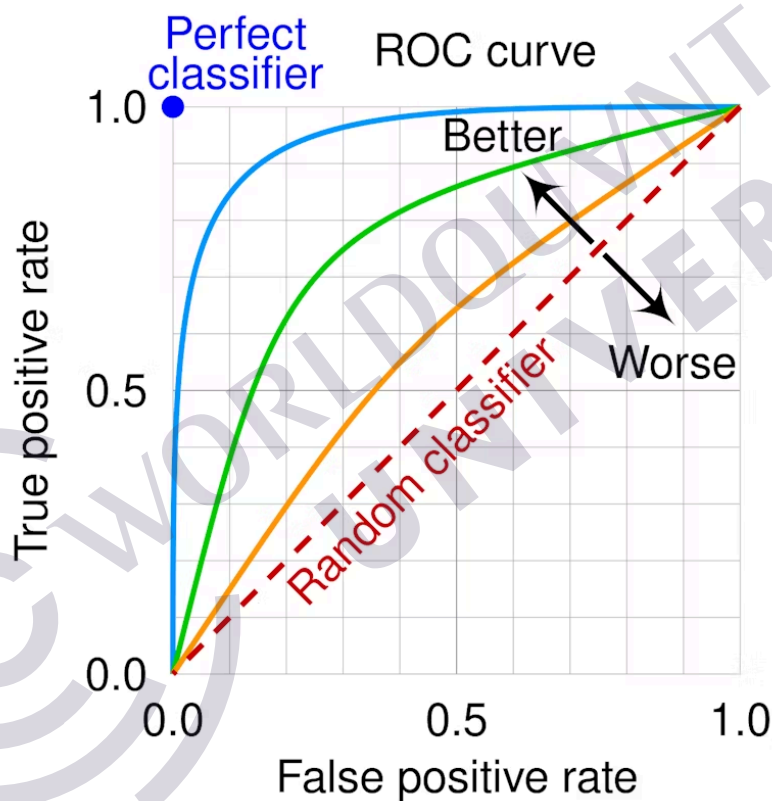
The components are **TN**, the true negatives, i.e., the correctly predicted negatives or class 0, whereas **FN** are the false negatives, i.e., the actual class 1s that are predicted as 0. Similarly, **TP** and **FP** are the true positives and false positives, respectively. Using the entries from the confusion matrix, we have the true positive (**TPR**) and false positive rates (**FPR**),

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Keep in mind that there is a threshold of probability that assigns a prediction to a class. As an example, should 50% be the threshold, any prediction with over 50% will be assigned class 1. Thus, **FPR** and **TPR** are sensitive to the threshold.

Figure 5: ROC Curve for Different Models



Source: CMG Lee, MartinThoma, <https://commons.wikimedia.org/w/index.php?curid=109730045>

The ROC is a plot of **TPR** against **FPR** for different thresholds and is shown in figure 5. Each line or series in the chart represents a model. The dashed red line is that of a model that is a random guess, whereas the blue line is that of an ideal model that predicts each class nearly perfectly, i.e., the off-diagonal elements of the confusion matrix are close to zero. We therefore want a model as close to the blue curve as possible. This is why the green curve represents a model that is better than the model represented by the orange curve. ROC enables us to visually see which model is better. However, a quantity that can be used to compare models or determine performance is the AUC.

7. Area Under the Curve (AUC)

The AUC is simply the area between the curve and the x-axis. The higher the AUC, the better the model at differentiating classes. You may still come across the Gini coefficient being used to measure a model's performance. However, keep in mind that the AUC and Gini are linearly related and synonymous since the relation established by Schechtman and Shapira (2) is,

$$Gini = 2 * AUC - 1.$$

For a more detailed explanation of the ROC curve with an example, watch the following video:

08:44

[Access video transcript here](#)

8. Conclusion

In this lesson, we looked at tree-based algorithms in classification and regression problems. We examined the methodology behind them and looked at boosting, which combines the power of many weak learners. A modified gradient boosting tree-based algorithm was also discussed as a metric for assessing model performance. In the next lesson, we apply tree-based algorithms to a financial problem.

References

- Gini, Corrado. "On the Measurement of Concentration and Variability of Characters." *Metron - International Journal of Statistics*, vol. 63, no. 1, 2005, pp. 1–38, <https://econpapers.repec.org/article/mtnancoec/0501.htm>. Originally published in Italian in *Atti del Reale Istituto Veneto di Scienze, Lettere ed Arti*, 1914.
- Hull, Isaiah. *Machine Learning for Economics and Finance in Tensorflow 2*. Apress, 2021. https://link.springer.com/chapter/10.1007/978-1-4842-6373-0_4.
- Schechtman, Edna, and Gideon Schechtman. (2016). "The Relationship between Gini Methodology and the ROC Curve." *METRON*, vol. 77, 2019, pp. 171–178, https://www.weizmann.ac.il/math/gideon/sites/math.gideon/files/uploads/AUC_Metron%20May%202019%20v4.pdf.